DESPLIEGUE DE APLICACIONES WEB.

Profesor: Miguel Ángel Rubiano Barros.

En él vamos a estudiar principalmente la instalación y configuración de los distintos tipos de servidores que pueden implicarse en el despliegue de aplicaciones. Utilizaremos servidores como **Apache** y **Tomcat** y se cubrirán tanto la instalación como la correcta configuración de los elementos.



Los contenidos de este documento están compartidos bajo una licencia Creative Commons si no se indica lo contrario. Todos los materiales usados para la confección de este documento están tomados de los apuntes de Sergio Cuesta Vicente y de los materiales del MEC para los Ciclos Formativos a distancia (proyecto avanza).

DESPLIEGUE DE APLICACIONES WEB.

Unidad 1. Implantación de arquitecturas web.

Profesor: Miguel Ángel Rubiano Barros.

Basado en material de Israel Caldito

Esta unidad didáctica sirve de introducción al módulo formativo. En ella, entre otros, se instalarán el servidor web Apache y el servidor de aplicaciones Tomcat.



Los contenidos de este documento están compartidos bajo una licencia Creative Commons si no se indica lo contrario. Todos los materiales usados para la confección de este documento están tomados de los apuntes de Sergio Cuesta Vicente y de los materiales del MEC para los Ciclos Formativos a distancia (proyecto avanza).

Tema 1. Implantación de arquitecturas web.

1. La arquitectura web y algunos modelos.

2. Un modelo simple para el despliegue de aplicaciones web.

2.1. ¿Qué es una aplicación web?

2.2. Fases de un proyecto de una aplicación web.

3. Servidores web.

3.1. ¿Qué necesito para montar un servidor web?

3.2. Alternativas.

3.3. Servicios web.

4. Instalación y configuración básica de un servidor web: Apache.

4.1. Instalación.

4.2. Comprobación y eliminación de otras versiones.

5. Instalar un servidor con LAMP.

Tema 1. Implantación de arquitecturas web.

6. Servidores de Aplicaciones.

6.1. Contenedores.

7. Tomcat.

7.1. Instalando Java.

7.2. Instalar Apache Tomcat.

7.3. Instalando paquetes adicionales.

7.4. Usuarios de Tomcat.

7.5. Iniciar y parar Tomcat.

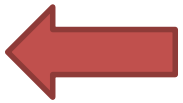
8. Aplicaciones web.

8.1. Estructura.

8.2. Descriptor de despliegue.

EMPEZAMOS

Tema 1. Implantación de arquitecturas web.

1. La arquitectura web y algunos modelos. 

2. Un modelo simple para el despliegue de aplicaciones web.

2.1. ¿Qué es una aplicación web?

2.2. Fases de un proyecto de una aplicación web.

3. Servidores web.

3.1. ¿Qué necesito para montar un servidor web?

3.2. Alternativas.

3.3. Servicios web.

4. Instalación y configuración básica de un servidor web: Apache.

4.1. Instalación.

4.2. Comprobación y eliminación de otras versiones.

5. Instalar un servidor con LAMP.

1. La arquitectura web y algunos modelos.

- Una aplicación web necesita de una estructura/esquema que permita su acceso desde diferentes lugares (máquinas).

- Esta estructura es lo que se denomina Arquitectura Web.

- En la actualidad, la gran mayoría de las arquitecturas web se basan en un modelo **cliente/servidor**: una comunicación asimétrica en la que uno de los extremos ofrece uno o más servicios y el otro hace uso de él .

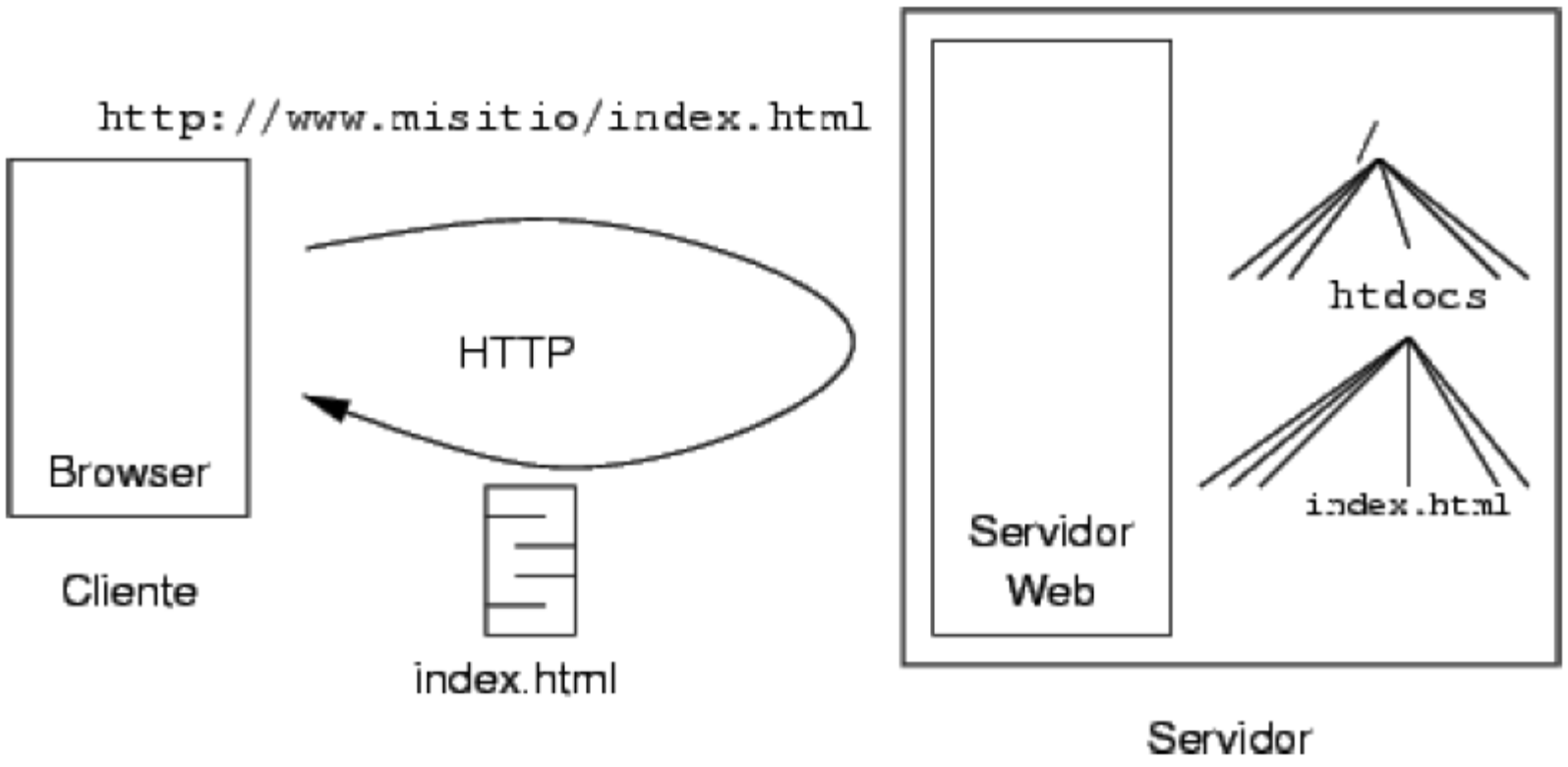
1. La arquitectura web y algunos modelos.

- Por tanto, una aplicación web es proporcionada por un **servidor web** y utilizada por los usuarios que se conectan desde cualquier punto a través de **clientes web** (browsers o navegadores).
- La arquitectura de un sitio web tiene tres componentes principales:
  - Un servidor web (p.e. Apache)
  - Una red de conexiones con protocolos (p.e. internet bajo TCP/IP)
  - Uno o más clientes (p.e. Mozilla Firefox)

1. La arquitectura web y algunos modelos.

- El servidor web distribuye páginas de información formateada a los clientes que las solicitan.
- Los requerimientos son hechos a través de una conexión de red, y para ello se usa el protocolo HTTP.
- Una vez que se solicita esta petición mediante el protocolo HTTP y la recibe el servidor web, éste localiza la página Web en su sistema de archivos y la envía de vuelta al navegador que la solicitó.

1. La arquitectura web y algunos modelos.



1. La arquitectura web y algunos modelos.

- Las aplicaciones web están basadas en el modelo **cliente/servidor** que gestionan servidores web y que utilizan como interfaz páginas web.
- Las páginas web son el componente principal de una aplicación o sitio web.
- Los browsers piden páginas (almacenadas o creadas dinámicamente) con información a los servidores Web.
- En algunos ambientes de desarrollo de aplicaciones Web, las páginas contienen código HTML y scripts dinámicos, que son ejecutados por el servidor antes de entregar la página.

1. La arquitectura web y algunos modelos.

- Una vez que se entrega una página, la conexión entre el browser y el servidor web se rompe, es decir que la lógica del negocio en el servidor solamente se activa por la ejecución de los scripts de las páginas solicitadas por el browser (en el servidor, no en el cliente).
- Cuando el browser ejecuta un script en el cliente éste no tiene acceso directo a los recursos del servidor.
- Los scripts del cliente son por lo general código JavaScript mezclados con código HTML.

1. La arquitectura web y algunos modelos.

- La colección de páginas son en una buena parte dinámicas (PHP, JSP, ASP, etc.), y están agrupadas lógicamente para dar un servicio al usuario.
- El acceso a las páginas está agrupado también en el tiempo (sesión).
- Los componentes de una aplicación Web son:

1. **Lógica del negocio.**
2. **Administración de los datos.**
3. **Interfaz.**

1. La arquitectura web y algunos modelos.

1. Lógica del negocio.

- Es la parte más importante de la aplicación.
- Definen los procesos que involucran a la aplicación.
- Conjunto de operaciones requeridas para proveer el servicio.

2. Administración de los datos.

- Manipulación de las Bases de Datos y archivos.

3. Interfaz.

- Los usuarios acceden a través de navegadores, móviles, tablets, etc.
- Funcionalidad accesible a través del navegador.
- Limitada y dirigida por la aplicación.

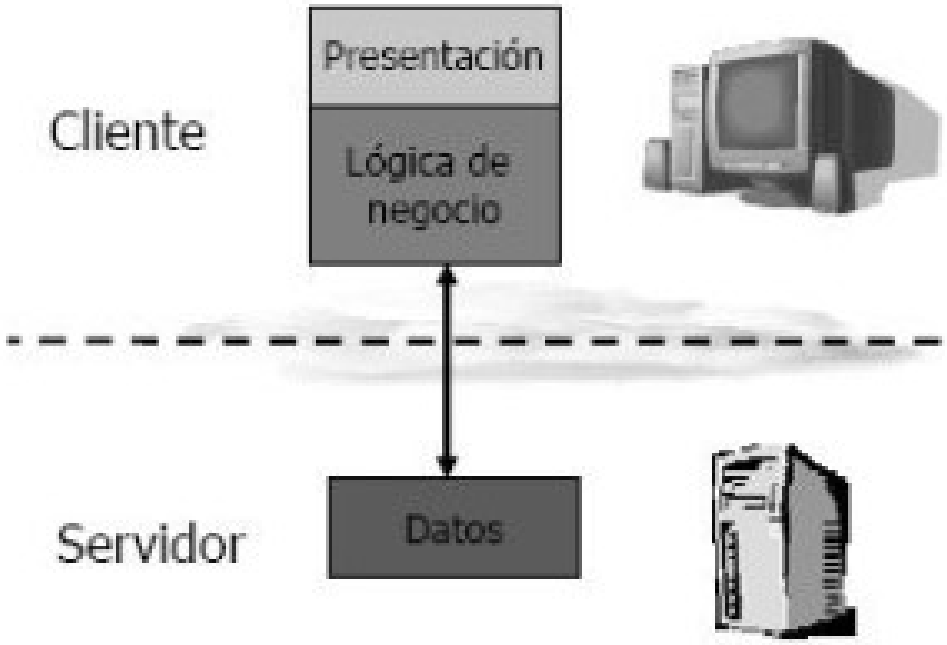
1. La arquitectura web y algunos modelos.

- Las aplicaciones web se modelan mediante lo que se conoce como **modelo de capas**.
- Una capa representa un elemento que procesa o trata información.
- Los tipos de modelos de capas son:
  - **Modelo de dos capas**: La información atraviesa dos capas entre la interfaz y la administración de los datos.
  - **Modelo de tres capas**: La información atraviesa tres capas: presentación, proceso y datos.

1. La arquitectura web y algunos modelos.

a) MODELO DE DOS CAPAS.

- Gran parte de la aplicación corre en el lado del cliente (fat client).
- Las capas son:
  - Cliente (fat client): la lógica de negocio está inmersa dentro de la aplicación que realiza el interfaz de usuario, en el lado del cliente.
  - Servidor: administra los datos.



1. La arquitectura web y algunos modelos.

b) MODELO DE TRES CAPAS.

- Esta diseñada para superar las limitaciones de las arquitecturas ajustadas al modelo de dos capas, introduce una capa intermedia (la capa de **proceso**).
- Entre **presentación** y los **datos**, los procesos pueden ser manejados de forma separada a la interfaz de usuario y a los datos, esta capa intermedia centraliza la lógica de negocio, haciendo la administración más sencilla, los datos se pueden integrar de múltiples fuentes.
- Las aplicaciones web actuales se ajustan a este modelo.

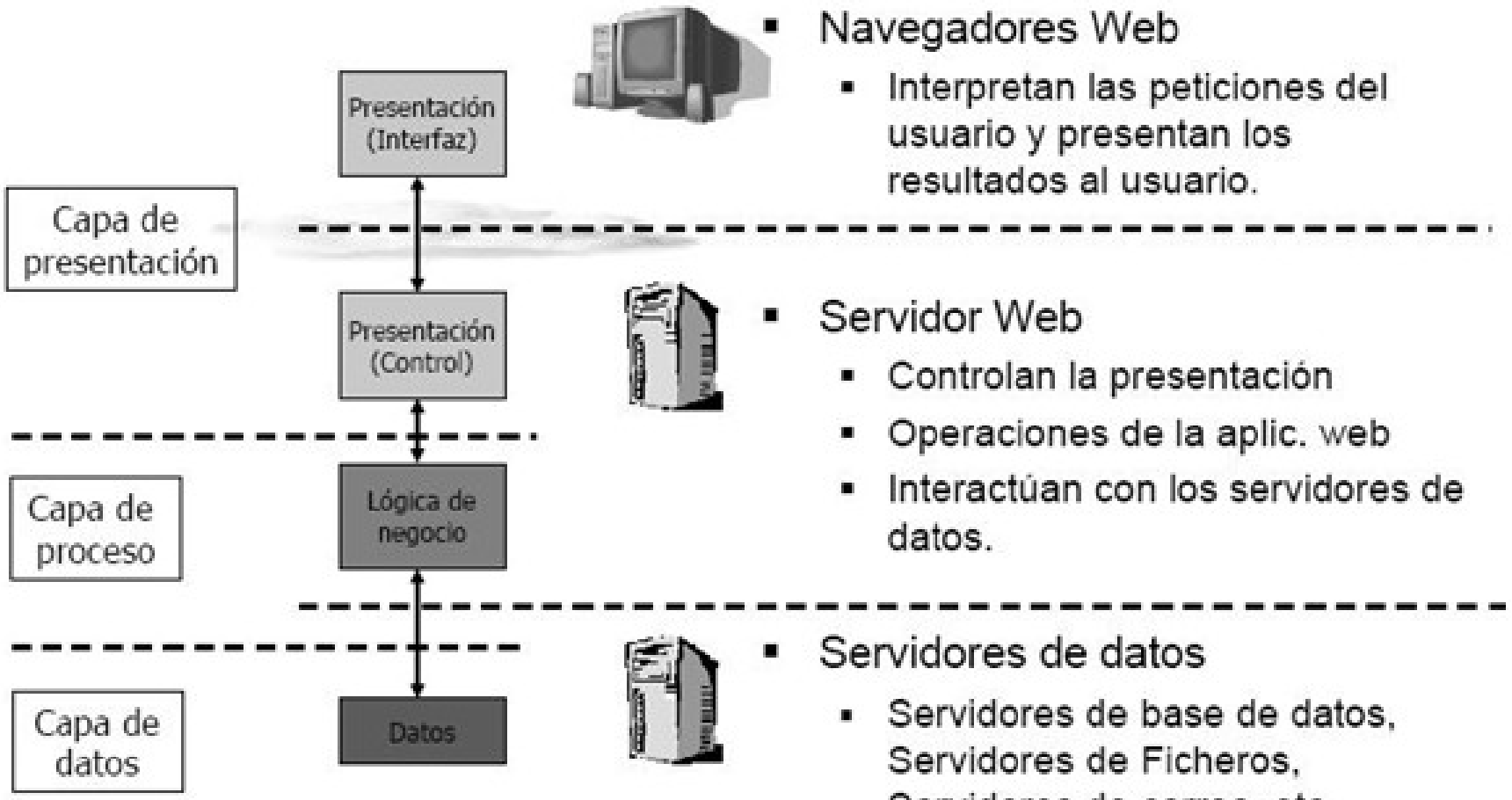
1. La arquitectura web y algunos modelos.

b) MODELO DE TRES CAPAS.

- Las capas de este modelo son:

1. **Capa de presentación** *(parte en el cliente y parte en el servidor).*
   - Recoge la información del usuario y la envía al servidor (cliente).
   - Manda información a la capa de proceso para su procesado.
   - Recibe los resultados de la capa de proceso y genera la presentación.
2. **Capa de proceso** *(servidor web).*
   - Recibe la entrada de datos de la capa de presentación.
   - Interactúa con la capa de datos para realizar operaciones.
   - Manda los resultados procesados a la capa de presentación.
3. **Capa de datos** *(servidor de datos).*
   - Almacena, recupera y mantiene los datos.
   - Garantiza la seguridad e integridad de los datos y usuarios.

1. La arquitectura web y algunos modelos.



- Navegadores Web
  - Interpretan las peticiones del usuario y presentan los resultados al usuario.
- Servidor Web
  - Controlan la presentación
  - Operaciones de la aplic. web
  - Interactúan con los servidores de datos.
- Servidores de datos
  - Servidores de base de datos, Servidores de Ficheros, Servidores de correo, etc..

1. La arquitectura web y algunos modelos.

- Resumiendo, la estructura de una arquitectura web actual sigue el siguiente modelo:
  1. Una **capa cliente**: es generalmente el navegador Web ejecutándose en el ordenador del usuario final.
  2. Un servidor Web (**capa de negocio**): La capa cliente puede acceder a diferente lógica y procedimientos que existen en la capa de negocio.

     Aquí la lógica puede ser mucho más compleja que en la capa anterior.

     Los componentes de esta capa pueden ser desde simples archivos HTML hasta Servlets de Java.

     Existen muchas tecnologías que pueden usarse en este nivel: por ejemplo scripting web como PHP, ASP o JSP a lenguajes de programación como TCL, CORBA y PERL.

1. La arquitectura web y algunos modelos.

3. Una **capa de datos**: Se compone de un sistema de almacenamiento acceso a datos que se utilizan para confeccionar la pagina Web.

Generalmente es un gestor de bases de datos relacionales (SGDB) pero pueden ser ficheros de texto plano, ficheros XML, etc.

Una opción cada vez más usada es la creación de ficheros XML a partir de datos almacenados en una base de datos.

1. La arquitectura web y algunos modelos.

- En el caso de estar usando paginas web estáticas no existiría la capa de datos ya que estos van incorporados en los propios archivos de marcas que serán las conforman las paginas web.

- Por otra parte, debido a la introducción de dinamismo en las paginas, la estructura vista anteriormente se ha visto alterada sensiblemente:

  - Los **navegadores Web** son capaces de interpretar diferentes elementos dinámicos autónomamente usando, por ejemplo, JavaScript.

  - Los **servidores Web** también pueden interpretar código para generar las paginas web.

    Así se pueden introducir pequeños programas que alteren el contenido o aspecto final de una pagina web dependiendo de diferentes elementos como el usuario que accede o la información solicitada en cada momento.

1. La arquitectura web y algunos modelos.

El servidor web necesita de algún modulo adicional para poder interpretar este código.

Generalmente se empotra en el propio servidor web para lenguajes de script (solución LAMP) o se incorpora en un servidor a parte (de aplicaciones) para los lenguajes más potentes (Tomcat en el caso de Java).

Algunos lenguajes que típicamente se usan en las paginas dinámicas en el servidor son PHP, Python, Ruby o Java.

Estos lenguajes también permiten el acceso a la capa de datos y la intercalación de estos datos entre los elementos de la pagina final (mezcla entre datos puros con códigos HMTL, CSS, PHP, Java, etc).

1. La arquitectura web y algunos modelos.

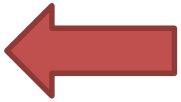
EJERCICIO 1

Investiga en internet a que se llama **Modelo Vista Controlador**. Resume el concepto con tus propias palabras y entrégalo en formato pdf.

Tema 1. Implantación de arquitecturas web.

1. La arquitectura web y algunos modelos.

2. Un modelo simple para el despliegue de aplicaciones web.



2.1. ¿Qué es una aplicación web?

2.2. Fases de un proyecto de una aplicación web.

3. Servidores web.

3.1. ¿Qué necesito para montar un servidor web?

3.2. Alternativas.

3.3. Servicios web.

4. Instalación y configuración básica de un servidor web: Apache.

4.1. Instalación.

4.2. Comprobación y eliminación de otras versiones.

5. Instalar un servidor con LAMP.

2. Un modelo simple para el despliegue de aplicaciones web.

- En la actualidad muchos negocios necesitan de internet para vender sus productos/servicios. *Por ejemplo, las tiendas online de todo tipo o la compra-venta de activos entre dos entidades bancarias (B2B).*

- **Aspectos clave**: la seguridad, la velocidad y la estabilidad de la comunicación y del servicio.

- Factores a tener en cuenta cuando se desarrolla y se quiere implantar posteriormente una aplicación web:
  - Hacerse una idea general de la aplicación y de las soluciones.
  - Tener en cuenta las tres capas estudiadas anteriormente.
  - No usar siempre las mismas tecnologías.
  - Considerar el coste y el presupuesto que tenemos.

Estos factores hay que tenerlo muy presentes antes de firmar ningún contrato e incluso antes de dar un presupuesto aunque sea orientativo.

2. Un modelo simple para el despliegue de aplicaciones web.

POR EJEMPLO:

*Una **compañía de venta de vehículos industriales** quiere una aplicación web para publicar sus datos de ventas y que los comerciales puedan acceder a ella remotamente.*

*Necesitaremos una **base de datos** en la que se almacenen los diferentes vehículos y sus ventas. También hará falta una **lógica** que mantenga todo el sistema actualizado y permita modificaciones.*

*Además necesitaremos una **capa de cliente con autenticación** para que los diferentes vendedores puedan acceder al sistema, consultar y actualizar los datos.*

Todas estas partes hay que diseñarlas y escribirlas en código usando los distintos lenguajes de programación disponibles para la Web.

***Después de la evaluación se puede decidir no afrontar el proyecto por muchos motivos**. Además de los costes ya mencionados podría darse el caso de que no tengamos los conocimientos o la infraestructura para llevar a cabo el proyecto.*

Tema 1. Implantación de arquitecturas web.

1. La arquitectura web y algunos modelos.

2. Un modelo simple para el despliegue de aplicaciones web.

2.1. ¿Qué es una aplicación web?

2.2. Fases de un proyecto de una aplicación web.

3. Servidores web.

3.1. ¿Qué necesito para montar un servidor web?

3.2. Alternativas.

3.3. Servicios web.

4. Instalación y configuración básica de un servidor web: Apache.

4.1. Instalación.

4.2. Comprobación y eliminación de otras versiones.

5. Instalar un servidor con LAMP.

2.1. ¿Qué es una aplicación web?

- Es una aplicación que se va a ejecutar a través de internet.
- Constará de dos partes (al menos) una en el lado servidor y otra que se ejecutará en la máquinas del cliente en un navegador web.
- Las aplicaciones web se engloban en el concepto superior de aplicaciones distribuidas.
- El servidor pone a disposición del cliente diferentes recursos.
- Ejemplos de aplicaciones web son el correo electrónico web, las tiendas online, las redes sociales, etc.

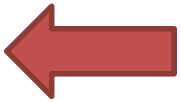
Tema 1. Implantación de arquitecturas web.

1. La arquitectura web y algunos modelos.

2. Un modelo simple para el despliegue de aplicaciones web.

2.1. ¿Qué es una aplicación web?

2.2. Fases de un proyecto de una aplicación web.



3. Servidores web.

3.1. ¿Qué necesito para montar un servidor web?

3.2. Alternativas.

3.3. Servicios web.

4. Instalación y configuración básica de un servidor web: Apache.

4.1. Instalación.

4.2. Comprobación y eliminación de otras versiones.

5. Instalar un servidor con LAMP.

2.2. Fases de un proyecto de una aplicación web.

1. **Concepto de la aplicación web**:
   - Obtener una idea clara y concreta de lo que quiere el cliente.
   - Idea general de cómo se llevará a cabo y de si es viable o no.
   - Hay que determinar las limitaciones reales con que nos podemos encontrar (por ejemplo, la conexión a internet en la zona puede no ser suficiente para obtener los resultados previstos) (otro ejemplo de problema puede ser que la tecnología necesaria sea demasiado cara)

   ES VITAL QUE AL TERMINAR ESTA FASE SE TENGA UNA DOCUMENTACIÓN QUE DEFINA CLARAMENTE LOS LÍMITES Y OBJETIVOS DEL PROYECTO.

2. **Diseño de la aplicación web:**
   - Da respuesta a la pregunta: ¿Cómo haremos la aplicación?

2.2. Fases de un proyecto de una aplicación web.

- Hay que concretar las tecnologías (tanto software como hardware) que usaremos y cómo se van a comunicar entre ellas.
- Determinar los distintos módulos que usaremos y sus interfaces.
- Realizar un plan de proyecto realista en el que se dividan las tareas y responsabilidades y se calculen los tiempos para cada elemento así como su secuencia y dependencias.

3. **Desarrollo de la aplicación web:**
   - Desarrollar la aplicación en sí haciendo pruebas tanto unitarias como de integración.
   - Gestionar una documentación del desarrollo y un control de versiones.
4. **Pruebas e implantación de la aplicación web:**
   - Antes de ponerlo en producción hay que probarlo intensivamente.
   - Tener en cuenta la comunicación de nuestra aplicación con otros sistemas informáticos.
   - Cuanto más se parezca el sistema de pruebas al real, mejor.
   - El último paso es la instalación y puesta en marcha del sistema.

2.2. Fases de un proyecto de una aplicación web.

Una fase común a todos los proyectos informáticos y que no se incluye aquí en el **mantenimiento**.

Este concepto incluye dos partes: el mantenimiento del servicio y, la corrección de errores y mejoras.

La **primera** consiste en asegurarse de que todo sigue funcionando y solucionar los posibles errores y "caídas" del servicio.

La **segunda** consiste en ampliar el proyecto. Ambos casos suelen considerarse contratos a parte y por ello no se incluyen en el ciclo.

2.2. Fases de un proyecto de una aplicación web.

- El orden correcto para el desarrollo es empezar de abajo a arriba.

- Es decir, **primero** la capa de datos, **luego** la de negocio y **por último** la presentación al cliente.

- Muchas veces se tiene la tentación de hacerlo al revés. Esto es debido a que no se han identificado bien las necesidades y objetivos de proyecto o no se ha realizado un diseño concreto.

- Es un error que nos conducirá a muchas más modificaciones y errores en nuestra aplicación.

Tema 1. Implantación de arquitecturas web.

1. La arquitectura web y algunos modelos.

2. Un modelo simple para el despliegue de aplicaciones web.

2.1. ¿Qué es una aplicación web?

2.2. Fases de un proyecto de una aplicación web.

3. Servidores web.

3.1. ¿Qué necesito para montar un servidor web?

3.2. Alternativas.

3.3. Servicios web.

4. Instalación y configuración básica de un servidor web: Apache.

4.1. Instalación.

4.2. Comprobación y eliminación de otras versiones.

5. Instalar un servidor con LAMP.

3. Servidores web.

- Es un programa o conjunto de ellos que proporciona un servicio a través de una red.

- La comunicación con el servidor suele hacerse mediante el protocolo **http** (*hypertext transfer protocol*).

- Muchas veces, servidor web se usa como referencia también al hardware que lo aloja, pero esto es inexacto porque el mismo hardware puede albergar muchas otras funcionalidades o puede darse el caso de que un mismo hardware contenga varios servidores web (a veces simulados).

3. Servidores web.

- El **objetivo** de un servidor web es proporcionar los medios para permitir la comunicación entre dos o más programas o grupos de software sin importar la tecnología usada para crear y operar cada uno de ellos.

- En la actualidad el servidor web más extendido con mucha diferencia es **Apache**. Por ello será en el que centraremos este curso. Existen muchos otros servidores web.

3. Servidores web.

- Los servidores web se engloban en un conjunto de sistemas más general que se denomina **modelo distribuido** porque el sistema no es unitario, está repartido entre diferentes máquinas o conjuntos de hardware.
- Este modelo tiene que afrontar algunos problemas que hay que tener siempre en cuenta:
  1. La latencia y poca fiabilidad del transporte (por ejemplo la red).
  2. La falta de memoria compartida entre las partes.
  3. Los problemas derivados de fallos parciales.
  4. La gestión del acceso concurrente a recursos remotos.
  5. Problemas derivados de actualizaciones de alguna/s de las partes.

3. Servidores web.

EJERCICIO 2

Investiga en internet que es un **servidor web**. Pon tres ejemplos indicando el/los sistemas operativos donde pueden ser ejecutados y la última versión disponible.

Tema 1. Implantación de arquitecturas web.

1. La arquitectura web y algunos modelos.

2. Un modelo simple para el despliegue de aplicaciones web.

2.1. ¿Qué es una aplicación web?

2.2. Fases de un proyecto de una aplicación web.

3. Servidores web.

3.1. ¿Qué necesito para montar un servidor web?

3.2. Alternativas.

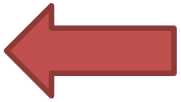
3.3. Servicios web.

4. Instalación y configuración básica de un servidor web: Apache.

4.1. Instalación.

4.2. Comprobación y eliminación de otras versiones.

5. Instalar un servidor con LAMP.



3.1. ¿Qué necesito para montar un servidor web?

- Lo primero que necesitas es una **máquina** con una potencia capaz de atender las peticiones que vaya a procesar.

Este punto es crítico y difícil de gestionar porque no sabemos cuál será la demanda y muchas veces es complejo estimar la carga de trabajo que soportará.

Es muy recomendable que sea una máquina dedicada o que cumpla otras funciones relacionadas con intercambio de información en internet como gestionar correo electrónico o FTP.

- También es vital que el **sistema operativo** que elijamos sea estable, tenga seguridad y control de permisos.

Los sistemas más habituales son diferentes versiones de UNIX (por ejemplo Solaris) pero las diferentes distribuciones de Linux están tomando una posición fuerte por su bajo (o inexistente) coste. Windows también es una buena opción (sobre todo sus versiones servidor) pero es más caro que Linux y es más difícil de gestionar debido a la diversidad de configuraciones, opciones y funcionalidad que lleven integrados.

3.1. ¿Qué necesito para montar un servidor web?

- Lo siguiente que tendrás que conseguir es una **dirección IP estática**.

Por supuesto debe ser una dirección de internet a no ser que tu objetivo sea montar una intranet.

Nuestra máquina debe ser accesible desde redes remotas.

EJERCICIO 3

Consulta cómo se puede contratar una dirección IP estática y cuánto cuesta.

3.1. ¿Qué necesito para montar un servidor web?

Los nombres y direcciones de internet que conocemos se basan en un sistema llamado DNS que lo que hace es convertir esas direcciones legibles para nosotros en direcciones IP y viceversa.

Si nuestra dirección IP cambia frecuentemente cuando alguien fuera a acceder a nuestra página esta le aparecería como no disponible a pesar de que todo el resto del sistema estuviera trabajando.

EJERCICIO 4.

¿Qué es DNS? ¿Quién lo gestiona? ¿Cómo se consigue un nombre de dominio?

Para alternativas como Wordpress o un hosting externo, ¿es necesario un nombre de dominio?

Existe la posibilidad de funcionar con una dirección IP dinámica mediante sistemas como http://dyn.com/dns/ que mantienen siempre actualizada nuestra dirección pero solo es recomendado (*incluso en la propia página*) para servidores con muy poca carga de conexiones y trabajo.

3.1. ¿Qué necesito para montar un servidor web?

- **Se necesita conexión a internet 24 horas** pero esto también implica ciertos problemas como la adquisición de dispositivos de red que aguanten ese horario sin sobrecalentarse o saturarse.

- Por supuesto todo el **software** necesario: sistema operativo, servidor web Apache, librerías ,etc...

- También **configurar** nuestra máquina para que sea accesible pero impidiendo la conexión de desconocidos a partes del sistema críticas o que no queremos publicar. En definitiva, hay que mejorar la seguridad.

Tema 1. Implantación de arquitecturas web.

1. La arquitectura web y algunos modelos.

2. Un modelo simple para el despliegue de aplicaciones web.

2.1. ¿Qué es una aplicación web?

2.2. Fases de un proyecto de una aplicación web.

3. Servidores web.

3.1. ¿Qué necesito para montar un servidor web?

3.2. Alternativas.

3.3. Servicios web.

4. Instalación y configuración básica de un servidor web: Apache.

4.1. Instalación.

4.2. Comprobación y eliminación de otras versiones.

5. Instalar un servidor con LAMP.

3.2. Alternativas.

CONSIDERACIONES PREVIAS:

Antes de decidirse a instalar nuestro propio servidor web, debemos tener en cuenta que no siempre es la mejor opción.

Lo primero que debemos saber es qué quiere el cliente.

Dependiendo del tamaño del servicio que vayamos a proporcionar y de la importancia de poder controlar todos los aspectos del servidor, podemos decidir usar otras posibilidades.

3.2. Alternativas.

Por otro lado la máquina que necesitamos tendrá que tener mucha RAM y capacidad de almacenamiento a parte de soportar grandes cargas de trabajo.

La conexión a internet también deberá ser potente y necesitaremos contratar una dirección IP estática.

Lo primero que se debe tener en cuenta es si nos interesa tener nuestro propio servidor web o contratar un servicio de almacenamiento web (web **hosting**).

Las ventajas de este último caso son obvias: no tenemos que preocuparnos de adquirir ni mantener ni el hardware ni el software necesario. Además la fiabilidad del servicio de una empresa especializada suele ser muy alta.

3.2. Alternativas.

Existen casos en los que incluso hay tecnologías más especificas para nuestras necesidades.

Cada vez es más habitual la existencia de sitios web en los que la apariencia no cambia pero el contenido es actualizado constantemente.

Para estos casos se puede usar un **gestor de contenidos**.

Con ellos se permite al usuario actualizar la información del sitio sin necesidad de que tenga conocimientos web concretos.

Existen muchos gestores web, algunos comerciales y caros pero muy ponentes. También hay dos alternativas gratuitas muy extendidas: ***Joomla*** *y* ***Drupal****.*

3.2. Alternativas.

Wordpress se considera incluso una alternativa más sencilla si se quiere una web informativa y bastante sencilla.

Wordpress empezó siendo una plataforma para alojar blogs pero cada vez incluye más opciones y posibilidades de configuración permitiendo llevar a cabo paginas bastante atractivas.

En la actualidad es un gestor de contenidos que incluso puede instalarse de manera autónoma.

Ejemplo de web hecha con wordpress: http://www.webdesignerdepot.com

3.2. Alternativas.

Ejercicio 5.

Apartado 1. Da tu opinión razonada sobre si es mejor montar tu propio servidor o contratar un hosting externo para alojarla. ¿Qué solución sería la más costosa? Pon algún ejemplo de hosting externo.

Apartado 2. Busca en Internet las principales características de los gestores de contenido como Joomla, Drupal o Wordpress y los motivos que te pueden llevar a decidirte por uno u otro.

Tema 1. Implantación de arquitecturas web.

1. La arquitectura web y algunos modelos.

2. Un modelo simple para el despliegue de aplicaciones web.

2.1. ¿Qué es una aplicación web?

2.2. Fases de un proyecto de una aplicación web.

3. Servidores web.

3.1. ¿Qué necesito para montar un servidor web?

3.2. Alternativas.

3.3. Servicios web.

4. Instalación y configuración básica de un servidor web: Apache.

4.1. Instalación.

4.2. Comprobación y eliminación de otras versiones.

5. Instalar un servidor con LAMP.

3.3. Servicios web.

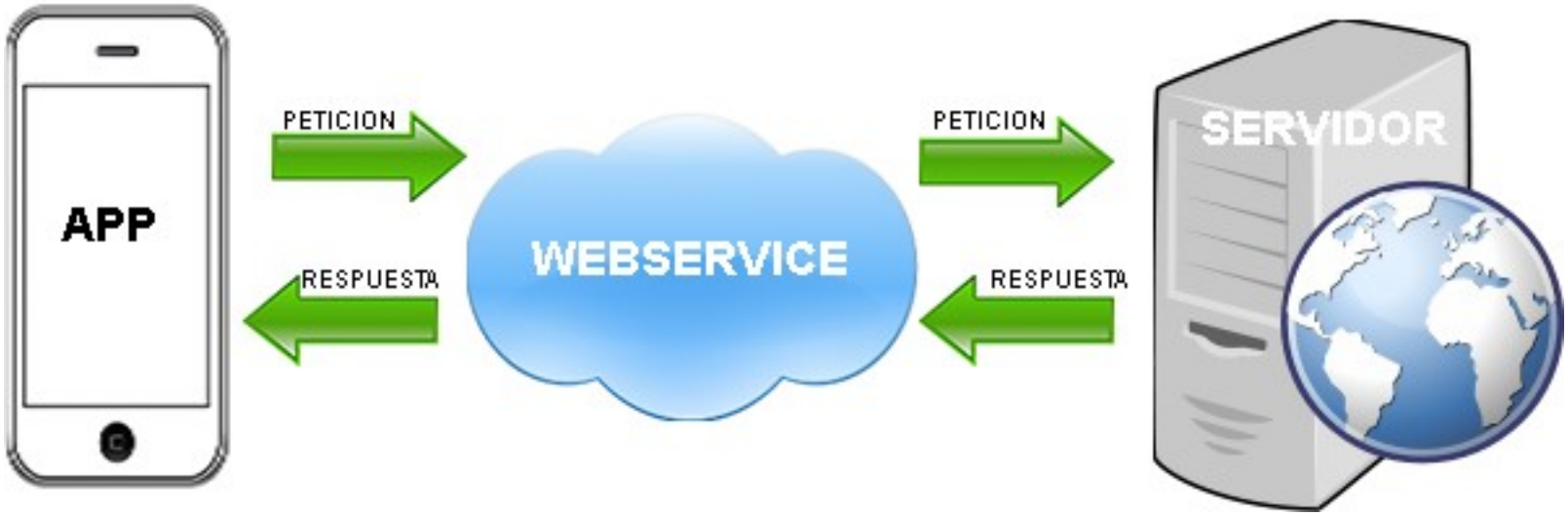
- Un **servicio web** es una tecnología que utiliza un conjunto de protocolos y estándares (SOAP, XML) que sirven para intercambiar datos entre aplicaciones a través de HTTP.
- Es, por tanto, un concepto más ambicioso que el de servidor web.



Distintas aplicaciones de software desarrolladas en lenguajes de programación diferentes, y ejecutadas sobre cualquier plataforma, pueden *utilizar los servicios web para intercambiar datos* en redes de ordenadores como internet.

3.3. Servicios web.

- Normalmente nos referimos con Servicio Web a una **colección de procedimientos (métodos) a los que podemos llamar desde cualquier lugar de Internet** o de nuestra intranet, siendo este mecanismo de invocación totalmente independiente de la plataforma que utilicemos y del lenguaje de programación en el que se haya implementado internamente el servicio.



3.3. Servicios web.

- Generalmente la definición de un servicio se realiza en una **API** que especifica cómo comunicarse con el servicio.



Tema 1. Implantación de arquitecturas web.

1. La arquitectura web y algunos modelos.

2. Un modelo simple para el despliegue de aplicaciones web.

2.1. ¿Qué es una aplicación web?

2.2. Fases de un proyecto de una aplicación web.

3. Servidores web.

3.1. ¿Qué necesito para montar un servidor web?

3.2. Alternativas.

3.3. Servicios web.

4. Instalación y configuración básica de un servidor web: Apache.

4.1. Instalación.

4.2. Comprobación y eliminación de otras versiones.

5. Instalar un servidor con LAMP.

4. Instalación y configuración básica de un servidor web: Apache.

- **Apache** destaca sobre otros por:
  - Diseño modular y altamente configurable.
  - Código abierto aunque existen extensiones y herramientas de terceros.
  - Funciona genial con **PHP**, **Perl** y otros lenguajes de script.
  - Existen versiones para muchos SOp: **Windows**, **Linux**, **Mac**, …

- La familia **Apache 2** mejoró a la primera en flexibilidad, escalabilidad y portabilidad.
- Lo lógico es instalarlo en un SOp de tipo servidor (Ubuntu Server) pero por motivos didácticos vamos a instalarlo en una versión estándar con interfaz gráfica pero desde la línea de comandos.

4. Instalación y configuración básica de un servidor web: Apache.

- **Apache** puede funcionar de forma autónoma, pero en la actualidad (más aún hablando de aplicaciones web) suele necesitar una BASE DE DATOS y cada vez son más los sistemas que usan PHP.
- Por ello han proliferado los instaladores **AMP** (Apache, MySQL y PHP) que instalan y configuran los tres sistemas para que funcionen de forma conjunta.
- ¡ESTO REQUIERE CONOCER EN PROFUNDIDAD LA CONFIGURACIÓN DE ESTOS TRES ELEMENTOS PARA PODER TENER UN SERVIDOR SEGURO!
- **XAMPP** (Apache + MySQL + PHP + Perl) es la versión más usada para Windows y Linux. La instalación de estos paquetes es muy sencilla.

4. Instalación y configuración básica de un servidor web: Apache.

- Antes de continuar, vamos a **preparar las herramientas** que usaremos a lo largo de este módulo **para hacer prácticas**.
- Me refiero a instalar:
  1. **Un gestor de máquinas virtuales**. Nosotros usaremos **`VirtualBox`** de Oracle.
     - URL: https://www.virtualbox.org/wiki/Downloads
  2. **Un sistema operativo del tipo Linux**. Nosotros optaremos por **`Ubuntu`** en su versión de escritorio.
     - URL:

EJERCICIO 6.

Descarga e instala la última versión de **VirtualBox** junto con su paquete de extensión. Haz capturas de pantalla para documentar y explicar el ejercicio.

4. Instalación y configuración básica de un servidor web: Apache.

- El siguiente paso es **crear una máquina virtual** del tipo Ubuntu (64 bits) con estas características:
  - Debe llamarse "**Ubuntu (Plantilla)**".
  - Ser del tipo "Linux".
  - La versión será "Ubuntu (64 bits)".
  - El tamaño de memoria debe ser 4096 MB.
  - Al menos poner dos núcleos.
  - El tipo de disco duro será VDI.
  - Establecer el tamaño del disco duro en 25 GB (reservado dinámicamente).

EJERCICIO 7.

Crea una máquina virtual con las características indicadas anteriormente. Es muy importante que se llame "**Ubuntu (Plantilla)**". Haz capturas de pantalla para documentar y explicar el ejercicio.

4. Instalación y configuración básica de un servidor web: Apache.

- El siguiente paso será **asociar la imagen de Ubuntu Desktop descargada con la máquina virtual** recientemente creada, para ello:
  - Sobre el icono de la máquina virtual pulsa con el botón derecho del ratón y elige la opción "*Configuración*".
  - Selecciona la pestaña "*Almacenamiento*".
  - Haz clic con el botón izquierdo sobre el controlador: IDE "*Vacío*". En el panel de la derecha aparecerá la información "*Atributos*" e "*Información*".
  - Pincha sobre el icono  de la unidad óptica "*IDE secundario maestro*" y selecciona la ISO del Ubuntu descargado. Pulsa aceptar.

EJERCICIO 8.

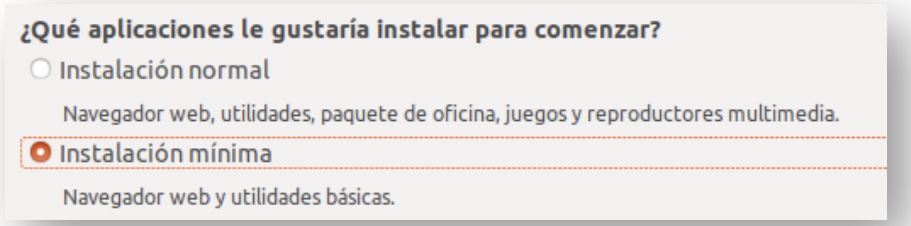
Asocia la máquina "Ubuntu (Plantilla)" con la ISO que descargaste de Ubuntu Desktop. Haz capturas de pantalla para documentar y explicar el ejercicio.

4. Instalación y configuración básica de un servidor web: Apache.

- Todavía no hemos terminado. Ahora tienes que **instalar Ubuntu Desktop en la máquina virtual**, para ello:
  - Haz doble clic sobre el icono de la máquina:



  - Sigue los pasos de instalación teniendo en cuenta que debes ***elegir una instalación mínima*** marcando además la opción de "*descargar actualizaciones al instalar Ubuntu*":

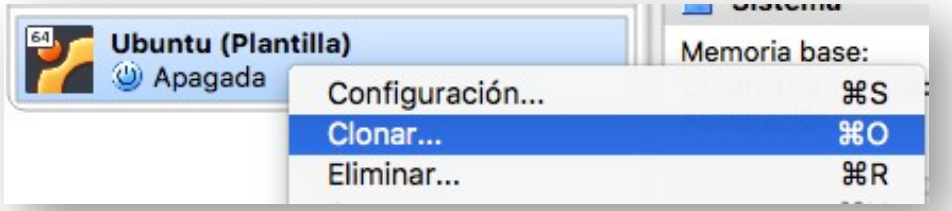


EJERCICIO 9.

Instala Ubuntu en la máquina virtual "Ubuntu (Plantilla)". Haz capturas de pantalla para documentar y explicar el ejercicio.

4. Instalación y configuración básica de un servidor web: Apache.

- Por último, apagamos la máquina virtual y la **clonamos**:
  - Sobre el icono de la máquina virtual pulsa con el botón derecho del ratón y elige la opción "*Clonar*".



  - Nombra a la nueva máquina "**Ubuntu (Apache)**". La usaremos en el siguiente apartado.
  - Elige como tipo de clonación "**Clonación completa**".

EJERCICIO 10.

Clona la máquina virtual "**Ubuntu (Plantilla)**" poniéndole al clon el nombre "**Ubuntu (Apache)**". Haz capturas de pantalla para documentar y explicar el ejercicio.

Tema 1. Implantación de arquitecturas web.

1. La arquitectura web y algunos modelos.

2. Un modelo simple para el despliegue de aplicaciones web.

2.1. ¿Qué es una aplicación web?

2.2. Fases de un proyecto de una aplicación web.

3. Servidores web.

3.1. ¿Qué necesito para montar un servidor web?

3.2. Alternativas.

3.3. Servicios web.

4. Instalación y configuración básica de un servidor web: Apache.

4.1. Instalación.

4.2. Comprobación y eliminación de otras versiones.

5. Instalar un servidor con LAMP.

4.1. Instalación



- Antes de instalar nuevo software es importante *tener actualizados los repositorios oficiales*:

```
$ sudo apt update
```

- También es buena costumbre subir, a las versiones más recientes, todo el software previamente instalado en nuestro Ubuntu:

```
$ sudo apt upgrade
```

- **Una vez hecho, ya estamos en disposición de instalar Apache**:

```
$ sudo apt install apache2
```

4.1. Instalación

- Lo primero que tenemos que hacer es ver que *ahora* **Apache** *está instalado en*:

  $ cd */etc/apache2/* **&& ls** -la

```
israel@daw:~$ cd /etc/apache2/ && ls -la
total 96
drwxr-xr-x   8 root root  4096 sep 20 21:31 .
drwxr-xr-x 121 root root 12288 sep 20 21:31 ..
-rw-r--r--   1 root root  7224 jun 27 19:05 apache2.conf
drwxr-xr-x   2 root root  4096 sep 20 21:31 conf-available
drwxr-xr-x   2 root root  4096 sep 20 21:31 conf-enabled
-rw-r--r--   1 root root  1782 jun 27 19:05 envvars
-rw-r--r--   1 root root 31063 jun 27 19:05 magic
drwxr-xr-x   2 root root 12288 sep 20 21:31 mods-available
drwxr-xr-x   2 root root  4096 sep 20 21:31 mods-enabled
-rw-r--r--   1 root root   320 jun 27 19:05 ports.conf
drwxr-xr-x   2 root root  4096 sep 20 21:31 sites-available
drwxr-xr-x   2 root root  4096 sep 20 21:31 sites-enabled
israel@daw:/etc/apache2$
```

4.1. Instalación

- Existen diferentes versiones de **Apache 2** por lo que es interesante comprobar la nuestra:

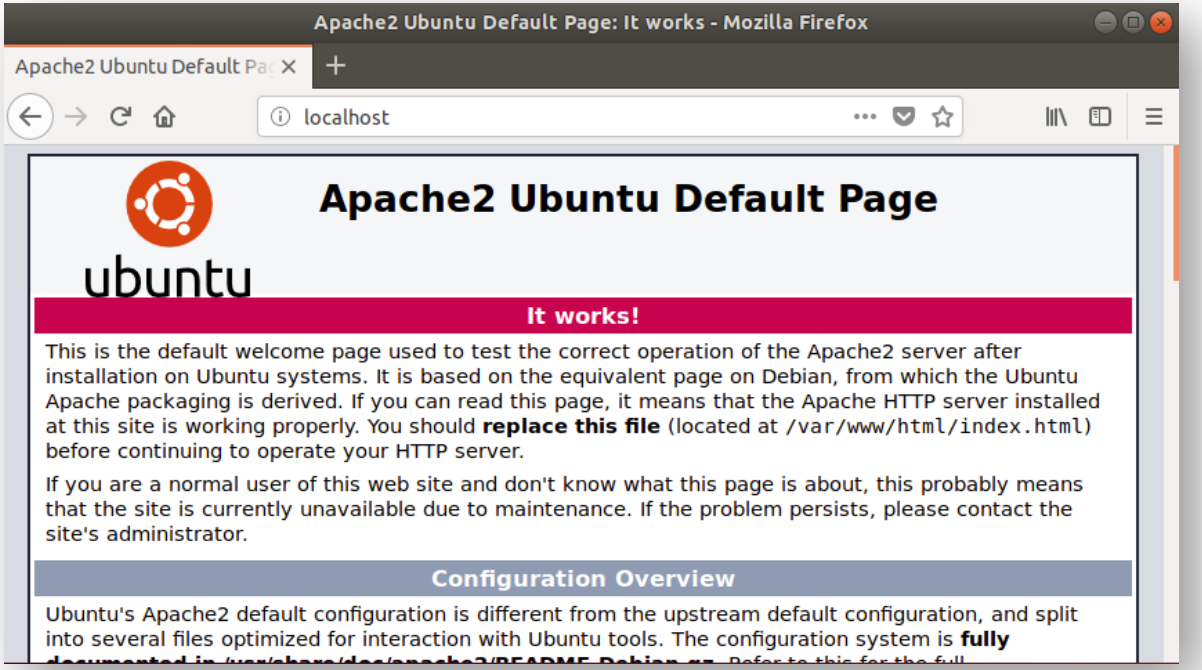
  $ apache2 *-v*

- Lo que nos mostrará algo como esto:

```
israel@daw:~$ apache2 -v
Server version: Apache/2.4.29 (Ubuntu)
Server built:   2018-06-27T17:05:04
israel@daw:~$
```

4.1. Instalación

- Para probar que Apache funciona iniciamos **Firefox** y escribimos: http://localhost



EJERCICIO 11.

Instala Apache en tu máquina virtual siguiendo los pasos indicados y comprueba el acceso desde la máquina virtual.

4.1. Instalación

- Si examinamos el contenido del directorio **/etc/init.d/** observamos que la instalación nos ha dejado un archivo llamado **apache2** que hace que se inicie el servidor web cada vez que arranquemos Ubuntu.
- Al editarlo:

```
$ sudo gedit /etc/init.d/apache2
```

- Nos muestra un contenido muy complejo. Si le echamos un vistazo por encima, podemos intuir que se usa para iniciar Apache en diversos modos (las órdenes más importantes son **start**, **stop**, **restart** y **graceful**).

EJERCICIO 12.

Echa un vistazo al script */etc/init.d/apache2* sin entrar en detalles. ¿Qué es **graceful**? ¿Para qué se usa aquí? Usa internet para documentar tu respuesta.

4.1. Instalación

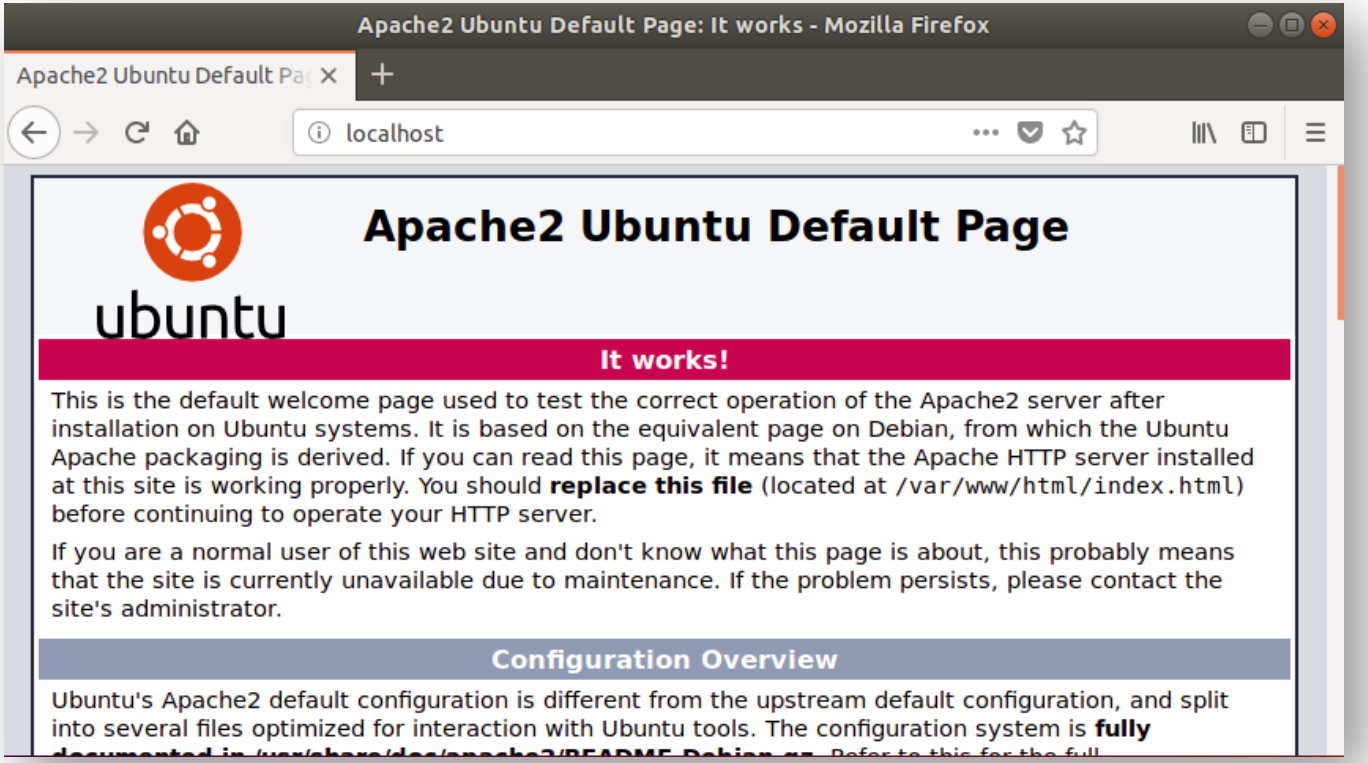
- Hay varias maneras de instalar **Apache**. Nosotros vamos a usar una configuración que utiliza la tecnología **DSO** (**Dynamic Shared Objects**).

- Esta configuración de Apache permite añadir más tarde módulos sin necesidad de recompilar todo el servidor. Por defecto en la instalación automática ya viene activada.

- La otra opción de instalación es bajar el código fuente de Apache junto con los módulos que se quieran utilizar y recompilar todo junto en un mismo bloque.

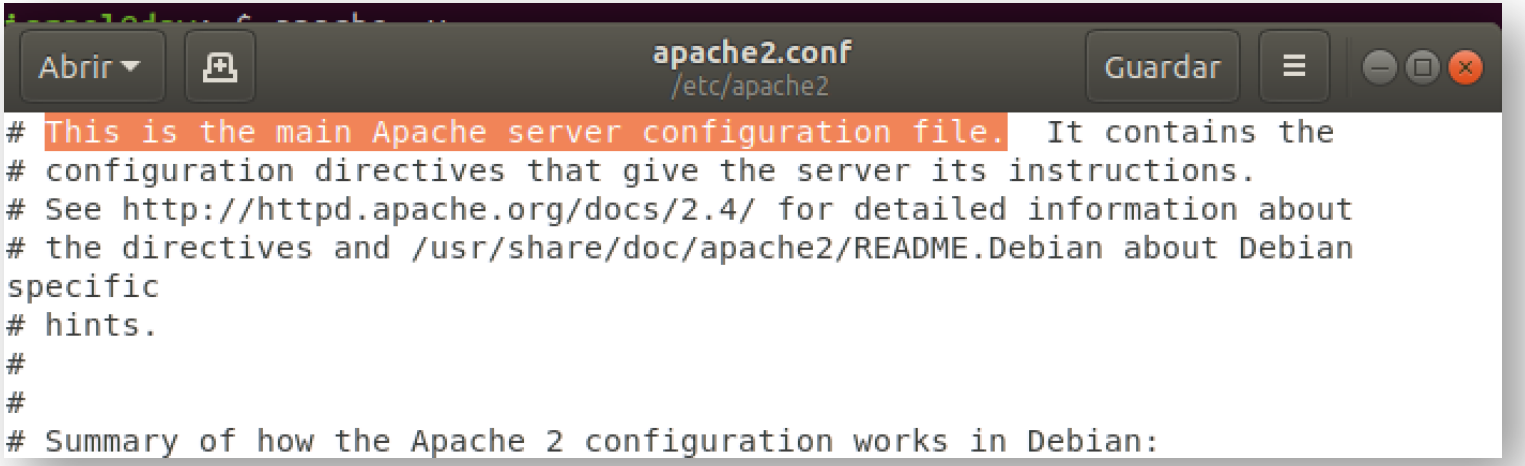
4.1. Instalación

- Ahora abrimos el navegador web y cargamos la página para ver que todo ha ido bien. Escribimos **http://localhost** y debemos encontrar la página que pone "It Works!".



4.1. Instalación

- En cuanto a la configuración de Apache, al realizar una **instalación estándar** (bajada de los repositorios de Ubuntu) estamos asumiendo que toda la configuración va centralizada en un único archivo,(*/etc/apache2/apache2.conf*) y desde éste se van llamando a otros ficheros más específicos.



4.1. Instalación

- Podemos además colocar algún **archivo HTML** (en /var/www/html/) y comprobar que se muestra en el navegador web.
- Por ejemplo, **http://localhost/saludo.html**



4.1. Instalación

- Aunque el espíritu de un servidor es estar activo todo el tiempo posible es importante poder **pararlo**, **arrancarlo** y **reiniciarlo** en ciertas situaciones. Todas estas opciones nos las explican en la documentación de Apache.

EJERCICIO 13.

Hemos hablado antes de la opción "**graceful**". ¿Qué sucede si usas "**apachectl -k graceful**"? ¿Es lógico? ¿Cuál es la diferencia con usar "**graceful-stop**"?

¿Cómo se debe parar el servidor para una operación de mantenimiento rutinario? ¿Y si lo que queremos es que se carguen algunas modificaciones en los archivos de configuración?

Tema 1. Implantación de arquitecturas web.

1. La arquitectura web y algunos modelos.

2. Un modelo simple para el despliegue de aplicaciones web.

2.1. ¿Qué es una aplicación web?

2.2. Fases de un proyecto de una aplicación web.

3. Servidores web.

3.1. ¿Qué necesito para montar un servidor web?

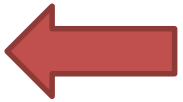
3.2. Alternativas.

3.3. Servicios web.

4. Instalación y configuración básica de un servidor web: Apache.

4.1. Instalación.

4.2. Comprobación y eliminación de otras versiones.



5. Instalar un servidor con LAMP.

4.2. Comprobación y eliminación de otras versiones

¡ANTENCIÓN!

Antes de estudiar este apartado vamos a clonar la máquina "*Ubuntu (Plantilla)*" creando una nueva que se llamará "*Ubuntu (LAMP)*". Asegúrate de tener cerrada la máquina "*Ubuntu (Plantilla)*" para poder clonarla.

Una vez hecho, te deben aparecer tres máquinas virtuales:

- Máquina virtual "**Ubuntu (Plantilla)**".
- Máquina virtual "**Ubuntu (Apache)**".
- Máquina virtual "**Ubuntu (LAMP)**".

Salta al apartado 5 de estos apuntes e instala LAMP sobre la máquina recién creada, es decir, **realiza el ejercicio 16**. Luego vuelve a este punto y continúa el apartado 4.2.

4.2. Comprobación y eliminación de otras versiones

- En la **versión de escritorio de Ubuntu** no vienen instalados los paquetes relativos a los servidores **Apache** y **MySQL**.
- Ni siquiera viene instalado el lenguaje de programación web **PHP**.
- Por tanto, si escribimos:

```
# dpkg --list | grep –e apache –e mysql –e php
```

- **No devolverá nada.**

EJERCICIO 14.

¿Qué hace la línea anterior? Busca en la red documentación que explique los comandos **dkpg** y **grep**.

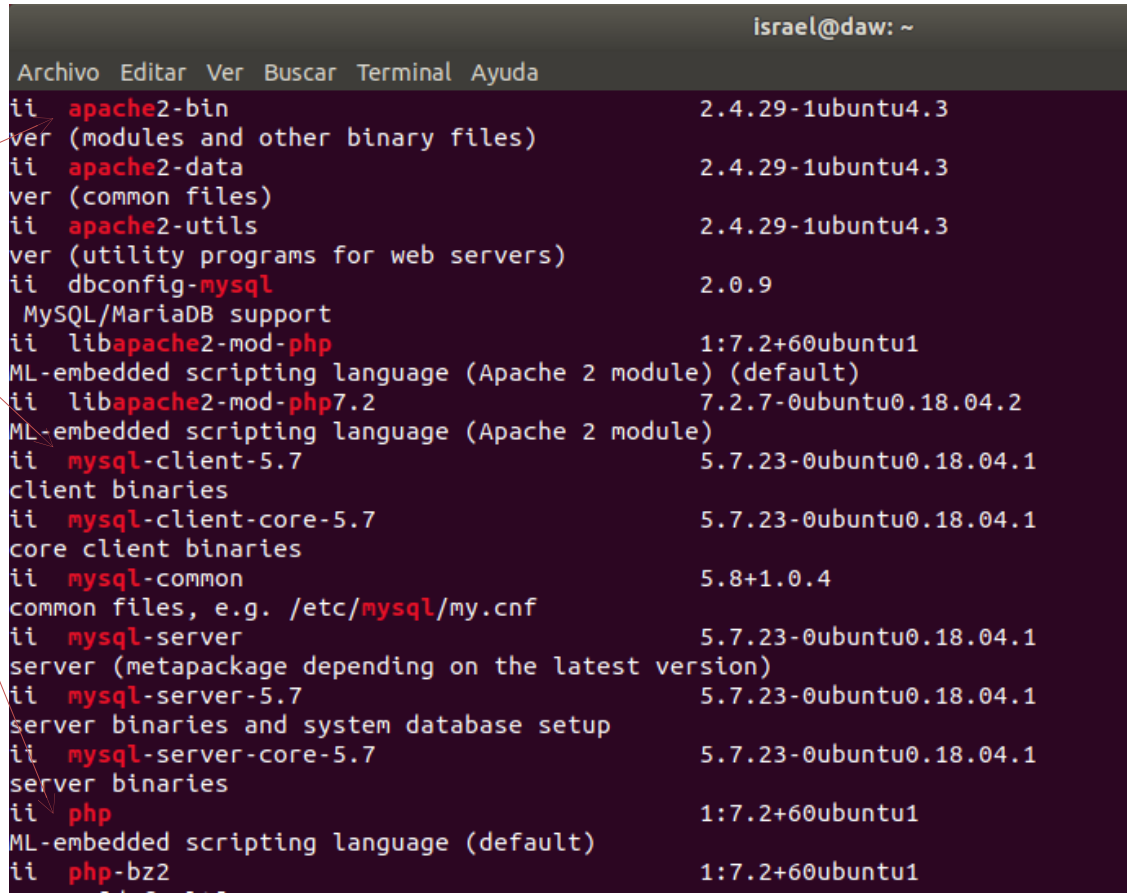
4.2. Comprobación y eliminación de otras versiones

- Sin embargo, es posible encontrarnos con instalaciones previas de **Apache**, **MySQL** y/o **PHP**; por ejemplo, si alguien instaló el paquete **LAMP** (cómo hemos hecho al comienzo de este apartado).
- En dichas situaciones, puede que nos resulte útil desinstalarlos para forzar una instalación limpia, así nos aseguraremos que tenemos las últimas versiones instaladas.
- Si volvemos a ejecutar el comando anterior sobre una instalación **LAMP**, veremos paquetes instalados relativos a **Apache**, **MySQL** y **PHP**.
- Lo vemos en la siguiente diapositiva:

4.2. Comprobación y eliminación de otras versiones

Ubuntu 18.04. Desktop con LAMP

Vemos como están instalados paquetes correspondientes a los servidores **Apache**, **MySQL** y **PHP**.



ii significa que está instalado el paquete.

4.2. Comprobación y eliminación de otras versiones

- Si queremos quitar **Apache**, **MySQL** y **PHP** tendríamos que hacer:

1. **Buscamos los procesos de Apache con los comandos ps y grep:**

```
$ ps -ef | grep apache2
```

Supongamos que nos muestra los siguientes procesos:

```
israel@daw:~$ ps -ef | grep apache2
root      13910     1  0 19:12 ?        00:00:00 /usr/sbin/apache2 -k start
www-data  13914 13910  0 19:12 ?        00:00:00 /usr/sbin/apache2 -k start
www-data  13915 13910  0 19:12 ?        00:00:00 /usr/sbin/apache2 -k start
www-data  13916 13910  0 19:12 ?        00:00:00 /usr/sbin/apache2 -k start
www-data  13917 13910  0 19:12 ?        00:00:00 /usr/sbin/apache2 -k start
www-data  13918 13910  0 19:12 ?        00:00:00 /usr/sbin/apache2 -k start
israel    14006  2971  0 19:16 pts/1    00:00:00 grep --color=auto apache2
```

2. **Paramos el servidor web Apache parando el servicio *apache2*:**

```
$ sudo service apache2 stop
```

4.2. Comprobación y eliminación de otras versiones

3. **Volvemos a buscarlo para asegurarnos de que ya no se está ejecutando:**

```
$ ps -ef | grep apache2
```

```
israel@daw:~$ sudo service apache2 stop
israel@daw:~$ ps -ef | grep apache2
israel    14720  2971  0 19:21 pts/1    00:00:00 grep --color=auto apache2
israel@daw:~$
```

4. **Buscamos ahora MySQL:**

```
$ ps -ef | grep mysql
```

Supongamos que nos muestra:

```
israel@daw:~$ ps -ef | grep mysql
mysql     12950     1  0 19:12 ?        00:00:00 /usr/sbin/mysqld --daemo
nize --pid-file=/run/mysqld/mysqld.pid
israel    15252  2971  0 19:33 pts/1    00:00:00 grep --color=auto mysql
israel@daw:~$
```

4.2. Comprobación y eliminación de otras versiones

5. Paramos MySQL:

```
# sudo /etc/init.d/mysql stop
```

```
israel@daw:~$ sudo /etc/init.d/mysql stop
[sudo] contraseña para israel:
[ ok ] Stopping mysql (via systemctl): mysql.service.
israel@daw:~$
```

Y comprobamos que ya no está:

```
# ps -ef | grep mysql
```

```
israel@daw:~$ ps -ef | grep mysql
israel   15408  2971  0 19:47 pts/1    00:00:00 grep --color=auto mysql
israel@daw:~$
```

4.2. Comprobación y eliminación de otras versiones

6. **Ahora que los servidores Apache y MySQL están parados podemos desinstalarlos** *(hay que tener en cuenta que PHP es una herramienta para hacer aplicaciones. No es un servidor, por eso no hay que pararlo)*:

```
$ dpkg -l | grep -e apache -e mysql -e php | awk '{print $2}' | xargs sudo dpkg --purge
```

```
israel@daw:~$ dpkg --list | grep -e apache -e mysql -e php | awk '{print $2}' | xargs sudo dpkg --purge
(Leyendo la base de datos ... 158234 ficheros o directorios instalados actualmente.)
Desinstalando apache2 (2.4.29-1ubuntu4.3) ...
Purgando ficheros de configuración de apache2 (2.4.29-1ubuntu4.3) ...
Desinstalando apache2-data (2.4.29-1ubuntu4.3) ...
Desinstalando apache2-utils (2.4.29-1ubuntu4.3) ...
Desinstalando libapache2-mod-php (1:7.2+60ubuntu1) ...
Desinstalando libapache2-mod-php7.2 (7.2.10-0ubuntu0.18.04.1) ...
Purgando ficheros de configuración de libapache2-mod-php7.2 (7.2.10-0ubuntu0.18.04.1) ...
dpkg: atención: al desinstalar libapache2-mod-php7.2, el directorio «/etc/php/7.2/apache2/conf.d» no es
tá vacío, por lo que no se borra
Desinstalando mysql-server (5.7.23-0ubuntu0.18.04.1) ...
Desinstalando mysql-server-5.7 (5.7.23-0ubuntu0.18.04.1) ...
update-alternatives: utilizando /etc/mysql/my.cnf.fallback para proveer /etc/mysql/my.cnf (my.cnf) en m
```

EJERCICIO 15.

¿Qué hace la línea anterior? Busca en la red documentación que explique los comandos **awk** y **xargs**. ¿Qué diferencias hay entre **--remove** y **--purge**?

4.2. Comprobación y eliminación de otras versiones

- **Ya está todo desinstalado.**
- Hay que insistir es que esto lo tendríamos que hacer solamente en situaciones excepcionales.
- Lo normal es que nos encontremos una instalación limpia y tengamos que instalar desde cero la infraestructura de servidores **Apache**, **MySQL** y el lenguaje de programación **PHP**.
- Es lo que vamos a estudiar en los siguientes apartados.
- Como ventaja tendremos una instalación limpia desde el principio.

Tema 1. Implantación de arquitecturas web.

1. La arquitectura web y algunos modelos.

2. Un modelo simple para el despliegue de aplicaciones web.

2.1. ¿Qué es una aplicación web?

2.2. Fases de un proyecto de una aplicación web.

3. Servidores web.

3.1. ¿Qué necesito para montar un servidor web?

3.2. Alternativas.

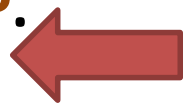
3.3. Servicios web.

4. Instalación y configuración básica de un servidor web: Apache.

4.1. Instalación.

4.2. Comprobación y eliminación de otras versiones.

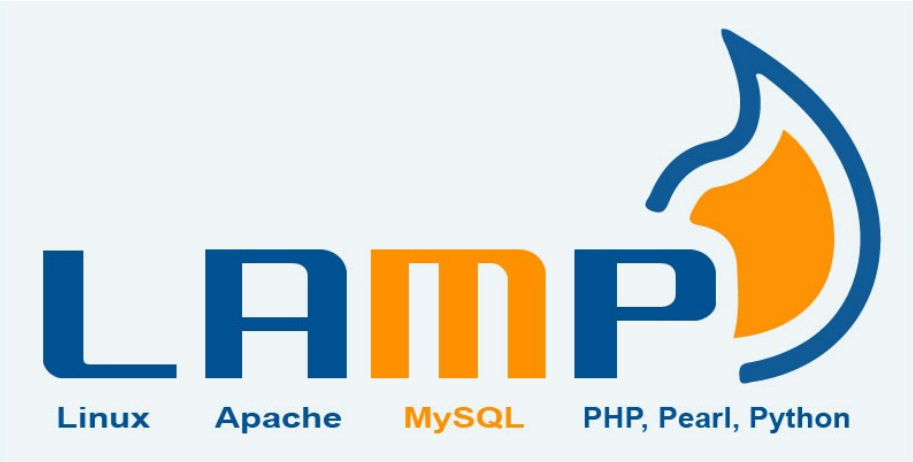
5. Instalar un servidor con LAMP.



5. Instalar un servidor con LAMP.

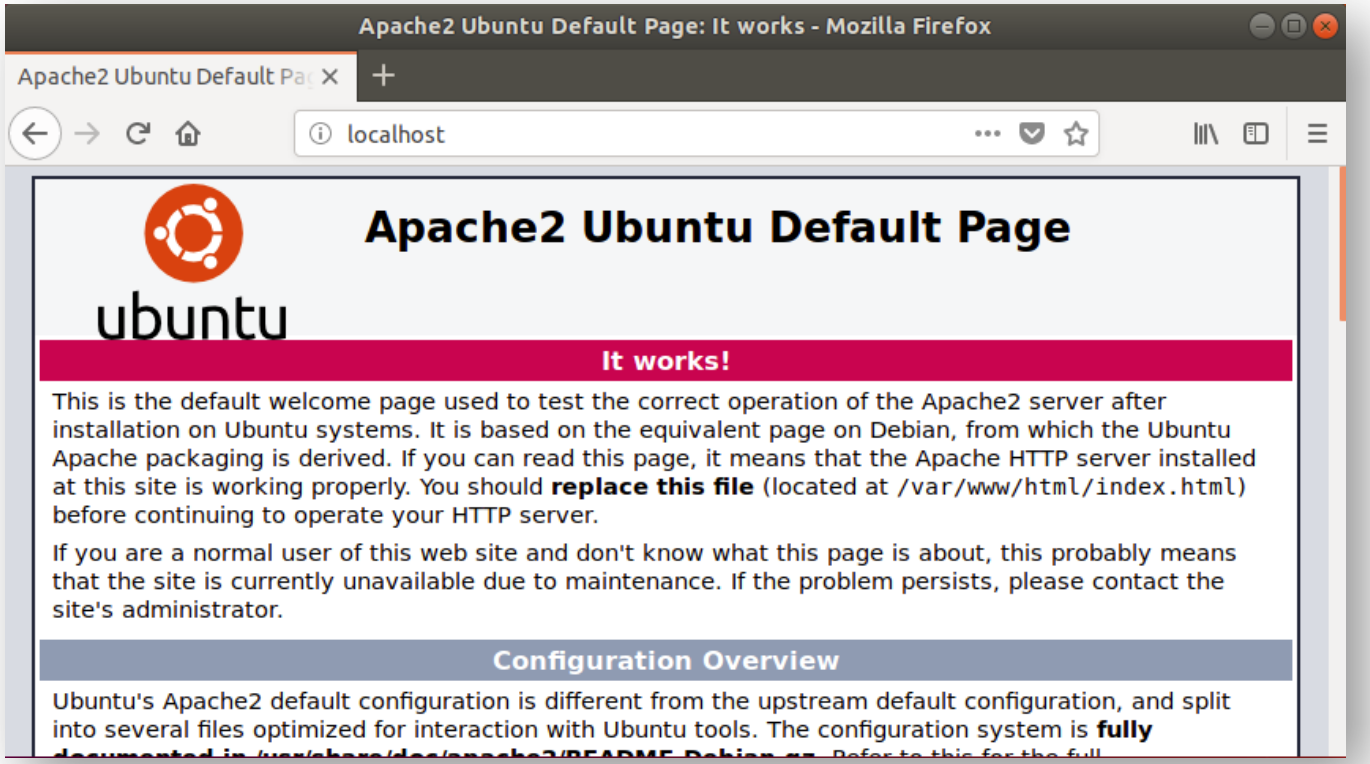
- Ahora vamos a empezar con una **nueva máquina virtual** a instalar un servidor con **Apache**, **MySQL** y **PHP** ya que como hemos comentado es la opción más habitual.
- Para ello lanzamos el comando:

```
$ sudo apt install lamp-server^
```



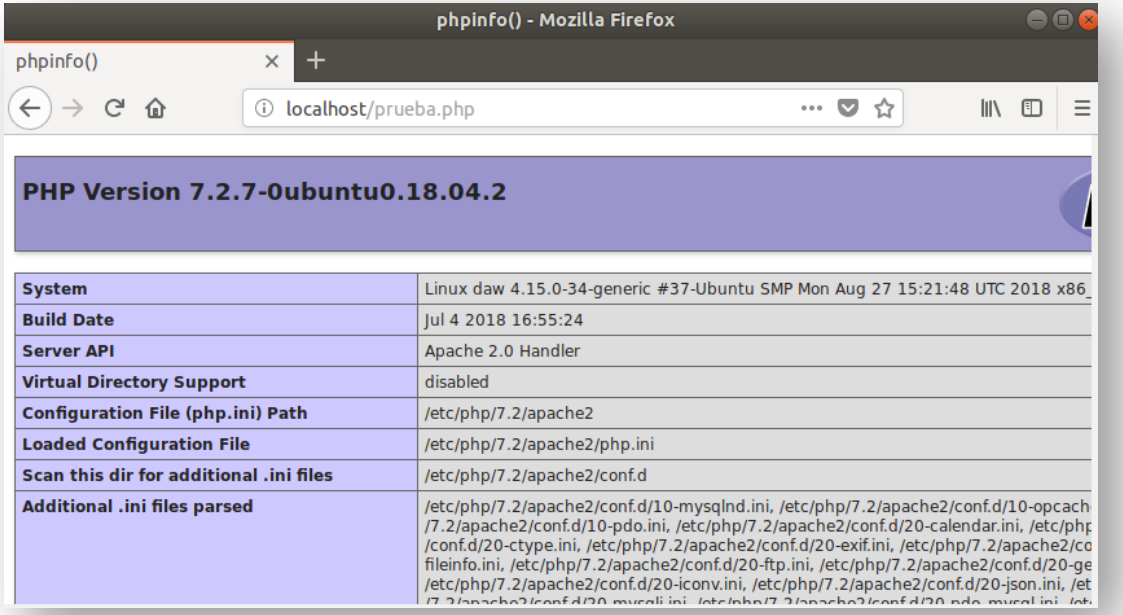
5. Instalar un servidor con LAMP.

- Para probar **Apache** simplemente abrimos el navegador y escribimos **localhost**:



5. Instalar un servidor con LAMP.

- Para probar **PHP**, creamos con `gedit` un archivo llamado `prueba.php` en la ruta */var/www/html* que solo contenga la línea: "`<?php phpinfo(); ?>`" (sin las comillas dobles).
- Abrimos el navegador y escribimos: `localhost/prueba.php`:



5. Instalar un servidor con LAMP.

- Para probar **MySQL** podemos usar uno de estos dos métodos:
  - o bien, conectarnos directamente,
  - o bien, instalar PHPMyAdmin.
- Si optamos por la primera opción deberemos escribir desde consola: **`sudo mysql -u root`**

```
israel@daw:~$ sudo mysql -u root
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 10
Server version: 5.7.23-0ubuntu0.18.04.1 (Ubuntu)

Copyright (c) 2000, 2018, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

Para salir del modo consola escribiremos **quit**

5. Instalar un servidor con LAMP.

- Si queremos instalar **PHPMyAdmin** escribimos:

```
$ sudo apt install phpmyadmin
```

- Una vez hecho, creamos un enlace simbólico de la carpeta recién instalada hacia la carpeta donde alojamos todas nuestras páginas web:

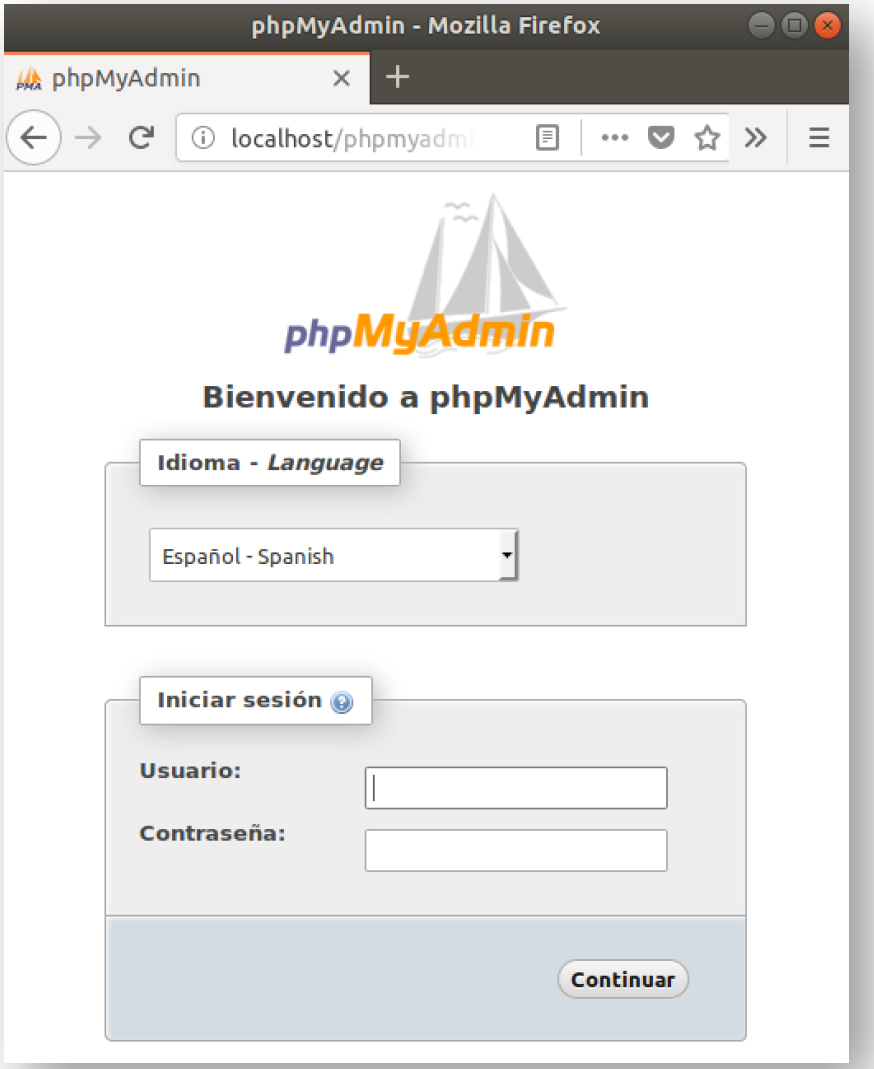
```
$ sudo ln -s /usr/share/phpmyadmin /var/www/html/
```

- Reiniciamos Apache

```
$ sudo service apache2 restart
```

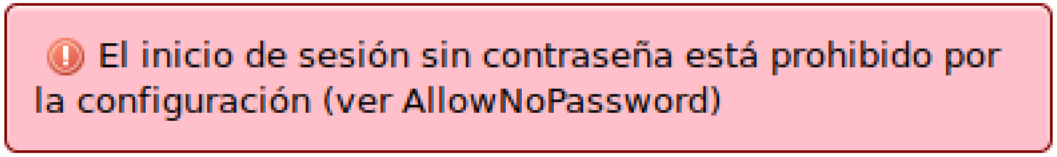
5. Instalar un servidor con LAMP.

- Y finalmente cargamos **phpMyAdmin** escribiendo en un navegador, por ejemplo en Firefox: **http://localhost/phpmyadmin**
- Para poder usar esta aplicación de gestión de bases de datos, necesitamos, lógicamente, un usuario registrado en **MySQL**.



5. Instalar un servidor con LAMP.

- Como hemos visto antes, para entrar en **phpMyAdmin**, necesitamos un usuario registrado previamente en **MySQL**:
  - Podemos entrar como *root* pero deberemos cambiar la *password*.
    - Ten presente que **tasksel** instala **MySQL** con *root* sin contraseña y **phpMyAdmin**, por motivos de seguridad, no permite el acceso en estas condiciones. (para cambiar la password de *root* debes hacer esto).



  - También podemos crear un nuevo usuario con contraseña que tenga similares permisos que *root*. Para ello entramos en modo consola en **MySQL** y escribimos (esto lo vimos el año pasado en BASES DE DATOS):

```
1 CREATE USER 'israel'@'localhost' IDENTIFIED BY 'israel';
2 GRANT ALL PRIVILEGES ON *.* TO 'israel'@'localhost' WITH GRANT OPTION;
3 FLUSH PRIVILEGES;
```

(Ahora ya podríamos entrar con el usuario: israel con password: israel).

5. Instalar un servidor con LAMP.

EJERCICIO 16.

Clona una nueva máquina virtual “Ubuntu (LAMP)” a partir de “Ubuntu (Plantilla)”.

Sobre la máquina virtual “Ubuntu (LAMP)” instala el servidor LAMP. Una vez hecho, prueba el correcto funcionamiento de todas las partes: Apache, PHP, MySQL y PHPMyAdmin. Crea un usuario en MySQL y comprueba que puedes entrar desde phpMyAdmin.

Tema 1. Implantación de arquitecturas web.

6. Servidores de Aplicaciones.

6.1. Contenedores.

7. Tomcat.

7.1. Instalando Java.

7.2. Instalar Apache Tomcat.

7.3. Instalando paquetes adicionales.

7.4. Usuarios de Tomcat.

7.5. Parar, iniciar y reiniciar Tomcat.

8. Aplicaciones web.

8.1. Estructura.

8.2. Descriptor de despliegue.

6. Servidores de aplicaciones.

- Un servidor de aplicaciones es un paquete software que proporciona servicios a las aplicaciones como pueden ser seguridad, servicios de datos, soporte para transacciones, balanceo de carga y gestión de sistemas distribuidos.
- El término se acuñó para servidores de la plataforma Java en su versión Enterprise Edition, pero en la actualidad se extiende a muchas otras tecnologías.
- Nosotros nos centraremos en **Tomcat**, un servidor de aplicaciones Java creado por Apache.
- Existen muchos otros como la integración de .NET en servidores de Microsoft, integración de PHP en un servidor para tener servidores de aplicaciones PHP, **Zend Server**, también para PHP, **Zope** para Python, **WebLogic** y **JBoss** para Java también, etc.

6. Servidores de aplicaciones.

- **Apache Tomcat** es un servidor de aplicaciones creado para alojar *Servlets* y *Java Server Pages* (JSP).
- Tomcat es gratuito y de código abierto pero no tiene nada que envidiar a otras soluciones comerciales.
- La versión que usaremos nosotros es la 10.
- El funcionamiento de un servidor de aplicaciones necesita de un servidor web. Muchas veces vienen en el mismo paquete, pero realmente son dos partes diferenciadas.
- Cuando un cliente hace una petición al servidor web, este trata de gestionarlo, pero hay muchos elementos con los que no sabe qué hacer. Aquí entra en juego **el servidor de aplicaciones**, que **descarga al servidor web de la gestión de determinados tipos de archivo**, en nuestro caso *servlets* y *JSP*.

6. Servidores de aplicaciones.

- Si un **cliente** hace una petición al **servidor** pidiendo un JSP, esta llega al servidor Web que lee un archivo XML que le proporciona el servidor de aplicaciones y determina que el archivo lo gestionará el servidor de aplicaciones.
- En el **archivo XML** también se incluye la dirección del servidor de aplicaciones y el servidor web le envía la petición mediante HTTP.

Tema 1. Implantación de arquitecturas web.

6. Servidores de Aplicaciones.

6.1. Contenedores.

7. Tomcat.

7.1. Instalando Java.

7.2. Instalar Apache Tomcat.

7.3. Instalando paquetes adicionales.

7.4. Usuarios de Tomcat.

7.5. Parar, iniciar y reiniciar Tomcat.

8. Aplicaciones web.

8.1. Estructura.

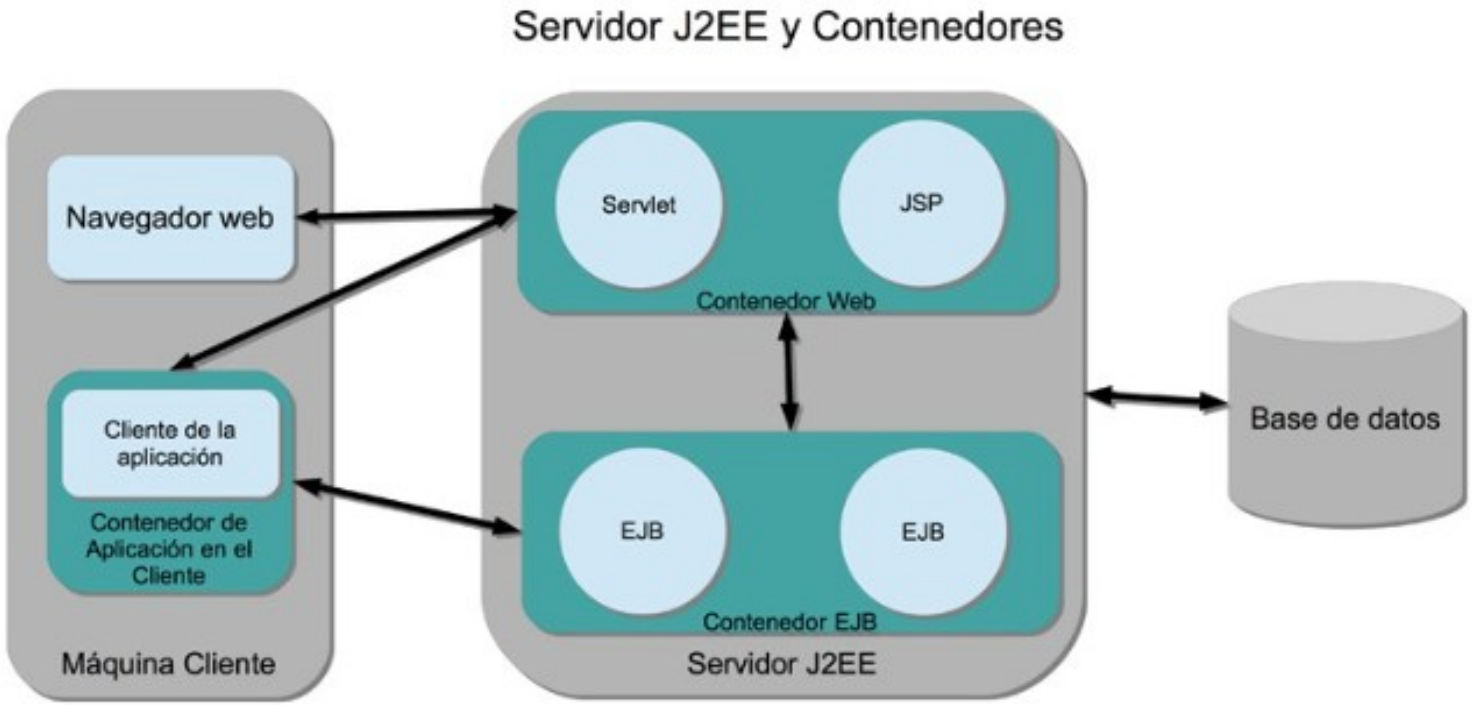
8.2. Descriptor de despliegue.

6.1. Contenedores

- El término *contenedor* es otro concepto bastante ambiguo como ya nos sucedió con *servidor*. En muchos casos se usa para referirse al propio servidor de aplicaciones e incluso al servidor web. Sin embargo, la acepción más extendida es otra.
- Los contenedores en los servidores de aplicaciones *son una forma de aislar la ejecución de cada aplicación o de cada **instancia** de una aplicación del resto de instancias y de otras aplicaciones*.
- Para cada ejecución proporcionan **seguridad**, **soporte para transacciones**, **conexión remota** y la **gestión de los recursos** precisos para la ejecución de la aplicación.

6.1. Contenedores

- La siguiente imagen muestra un esquema de **la estructura de contenedores** de Tomcat:



EJB – Enterprise JavaBeans (API's)

6.1. Contenedores

- En referencia a la imagen anterior, conviene concretar algunos conceptos:
  - El contenedor web se encarga de la ejecución de *servlets* y *JSPs* en un entorno controlado.
  - El contenedor Enterprise JavaBeans (EJB) se encarga de la ejecución de los *EJBs* (API's para construir aplicaciones empresariales JEE).
  - El **servidor J2EE** es el programa que proporciona contenedores EJB y Web.
  - El contenedor del cliente de la aplicación encarga de la ejecución de los componentes de las aplicaciones en la máquina del cliente.
  - El contendor de applets se encarga de ejecutar los applets en el clientes. Está compuesto por un navegador web y un plugin Java.

LOS CONTENEDORES DE TOMCAT SE DENOMINAN **CATALINA**.

Tema 1. Implantación de arquitecturas web.

6. Servidores de Aplicaciones.

6.1. Contenedores.

7. Tomcat

7.1. Instalando Java.

7.2. Instalar Apache Tomcat.

7.3. Instalando paquetes adicionales.

7.4. Usuarios de Tomcat.

7.5. Parar, iniciar y reiniciar Tomcat.

8. Aplicaciones web.

8.1. Estructura.

8.2. Descriptor de despliegue.

7. Tomcat.

- **Tomcat** es un servidor de aplicaciones que puede funcionar por si mismo.
- De hecho es capaz de procesar peticiones en HTTP y servir archivos HTML con bastante eficiencia, pero no tan bien como lo hace Apache.
- Generalmente si lo que queremos es un servidor web con funcionalidad adicional lo mejor es disponer de ambos servidores trabajando conjuntamente, pero para casos donde casi todo va a ser lógica en java con Tomcat funcionando autónomamente es suficiente.

7. Tomcat.

- En este primer acercamiento a Tomcat lo instalaremos en la máquina virtual llamada "*Apache*".
- Más adelante veremos como integrarlo con Apache para que cada cual se encargue de hacer lo que gestiona mejor.
- En este segundo caso, Apache recibirá todas las peticiones y enviará a Tomcat lo que le corresponda a él.
- En una máquina virtual de Java, solo puede ejecutarse una instancia de Tomcat.

EJERCICIO 17.

¿Qué hace la máquina virtual de Java? ¿En qué se diferencia de las máquinas virtuales que se crean con programas como Oracle Virtual Box o VMware?

Tema 1. Implantación de arquitecturas web.

6. Servidores de Aplicaciones.

6.1. Contenedores.

7. Tomcat.

7.1. Instalando Java.

7.2. Instalar Apache Tomcat.

7.3. Instalando paquetes adicionales.

7.4. Usuarios de Tomcat.

7.5. Parar, iniciar y reiniciar Tomcat.

8. Aplicaciones web.

8.1. Estructura.

8.2. Descriptor de despliegue.

7.1. Instalando Java

- Existen **dos versiones** de Java muy extendidas en entornos Linux.
- Una versión es la oficial de Oracle y la otra, de código abierto, se denomina OpenJDK.
- Ambos JDK pueden estar instalados en Ubuntu pero solamente una puede estar activa (en uso).
- **JDK** significa **Java Development Kit** y **JRE** significa **Java Runtime Environment**.
- Al instalar el JDK estamos instalando implícitamente el JRE. A la inversa no tiene porque cumplirse, de hecho no se cumple en la mayoría de las ocasiones ya que solamente instalaremos el JRE en los ordenadores clientes que no necesitan desarrollar programas escritos con tecnología Java.

7.1. Instalando Java

- **Nosotros solamente instalaremos y usaremos la última versión oficial del OpenJDK para distribuciones Ubuntu.**

OpenJDK:

- Esta versión es muy recomendable cuando se va a utilizar con otros sistemas de código abierto o de software libre como por ejemplo distribuciones **Linux Debian**.
- Para instalar la versión **OpenJDK** por defecto debemos, primero, actualizar los repositorios:

```
$ sudo apt update
```

- Seguidamente instalamos los paquetes default-jre y default-jdk:

```
$ sudo apt install default-jre default-jdk
```

7.1. Instalando Java

- Comprobamos la versión de Java:

  $ java *-version*

- Lo que debería mostrar algo como:

```
israel@ubuntu:~/Escritorio$ java -version
openjdk version "21.0.4" 2024-07-16
OpenJDK Runtime Environment (build 21.0.4+7-Ubuntu-1ubuntu224.04)
OpenJDK 64-Bit Server VM (build 21.0.4+7-Ubuntu-1ubuntu224.04, mixed mode, sharing)
```

- Por tanto, ahora tenemos la certeza de haber instalado el **OpenJDK** y ya podremos invocar al compilador de java desde cualquier lugar.

7.1. Instalando Java

EJERCICIO 18.

Renombra la maquina virtual “**Ubuntu (Apache)**” por “**Ubuntu (Apache+Tomcat)**” e instala la última versión de Java **OpenJDK**. Documenta el ejercicio con capturas.



Tema 1. Implantación de arquitecturas web.

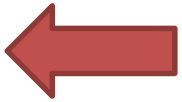
6. Servidores de Aplicaciones.

6.1. Contenedores.

7. Tomcat.

7.1. Instalando Java.

7.2. Instalar Apache Tomcat.



7.3. Instalando paquetes adicionales.

7.4. Usuarios de Tomcat.

7.5. Parar, iniciar y reiniciar Tomcat.

8. Aplicaciones web.

8.1. Estructura.

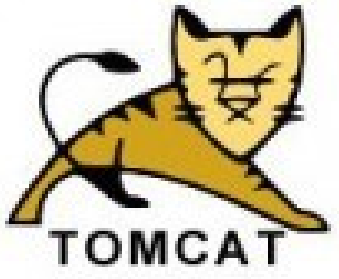
8.2. Descriptor de despliegue.

7.2. Instalar Apache Tomcat

- Para el correcto funcionamiento de **Tomcat**, es NECESARIO tener **Java** instalado y configurado adecuadamente.
- Los pasos son los mismos que ya vimos en el apartado "Instalando Java".
- Para instalar **Tomcat** haremos:

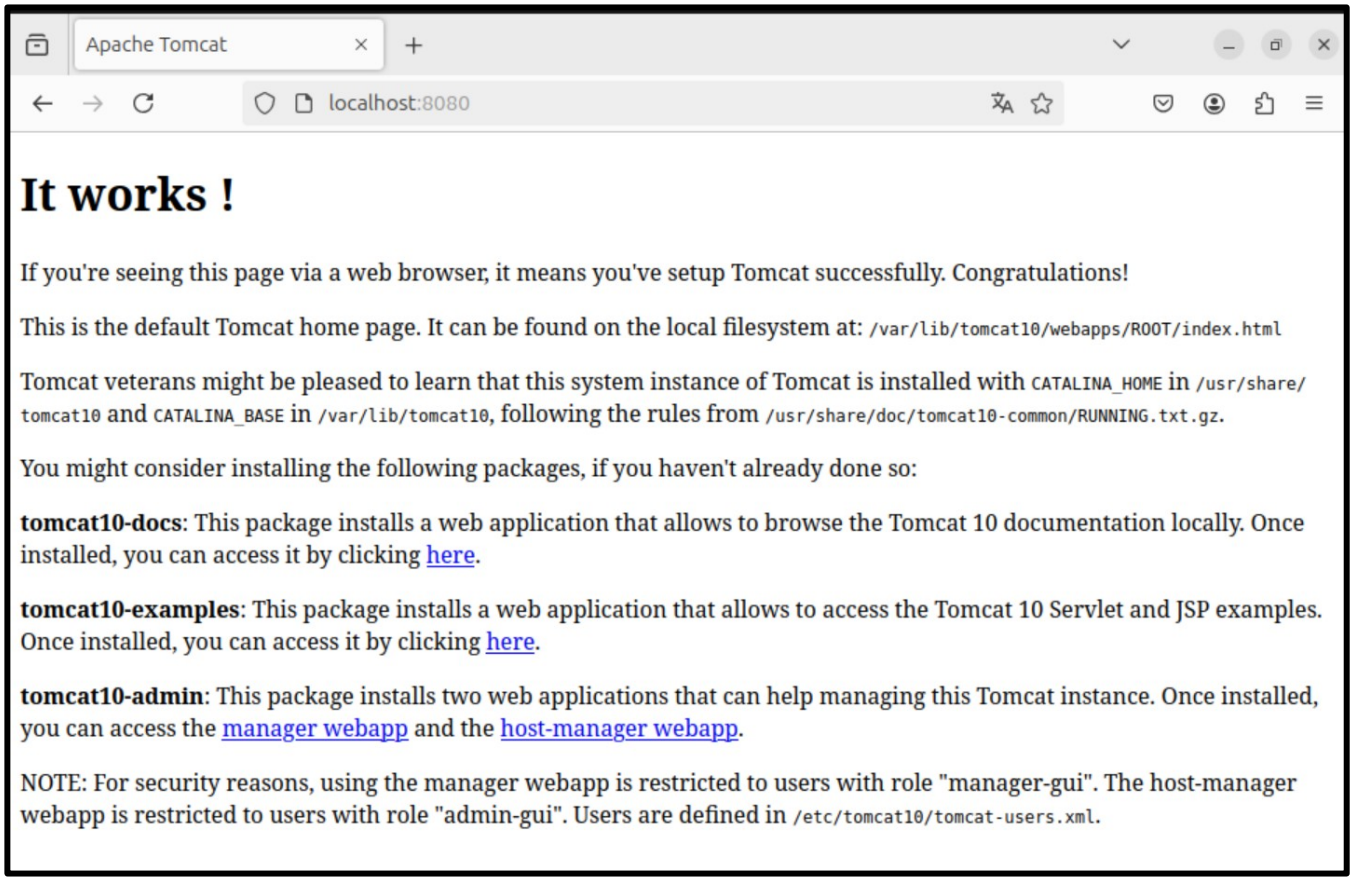
```
$ sudo apt install tomcat10
```

NOTA: En algunos casos, Tomcat no va a funcionar hasta que no configuremos sus variables de entorno: $CATALINA_BASE y $CATALINA_HOME.



7.2. Instalar Apache Tomcat

- Ahora podemos abrir un navegador web y acceder a: **http://localhost:8080.**
- Aparecerá algo similar a esto:



7.2. Instalar Apache Tomcat

- En el que nos indican la ubicación de dos variables de entorno muy necesarias en la configuración y uso de **Tomcat**: **CATALINA_HOME** y **CATALINA_BASE**.

> Tomcat veterans might be pleased to learn that this system instance of Tomcat is installed with `CATALINA_HOME` in `/usr/share/tomcat10` and `CATALINA_BASE` in `/var/lib/tomcat10`, following the rules from `/usr/share/doc/tomcat10-common/RUNNING.txt.gz`.

- Vamos a establecerlas como variables de entorno:

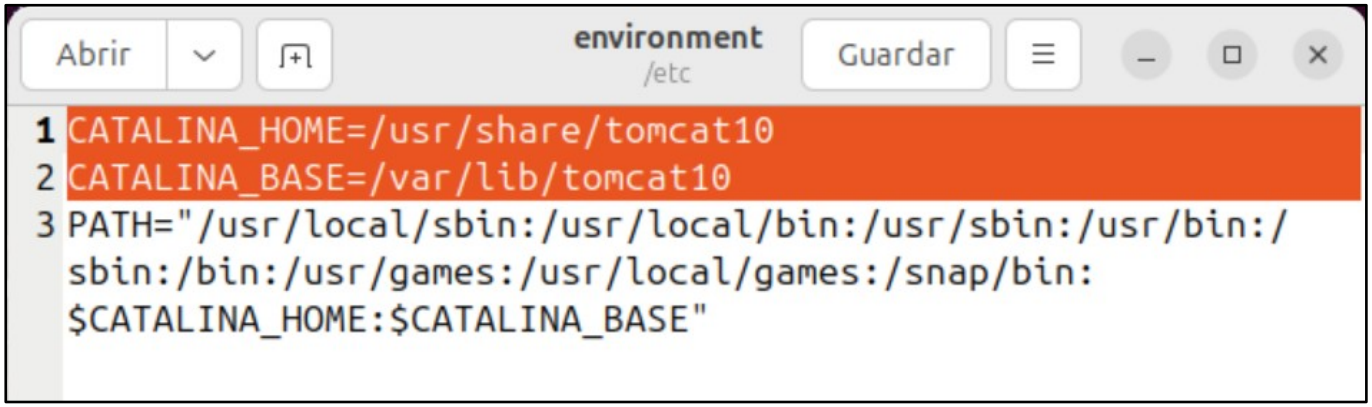
```
$ sudo gedit /etc/environment
```

7.2. Instalar Apache Tomcat

- Escribimos dichas variables:

```
CATALINA_HOME=/usr/share/tomcat10
CATALINA_BASE=/var/lib/tomcat10
```

- Y las añadimos al **PATH** que quedaría como sigue:

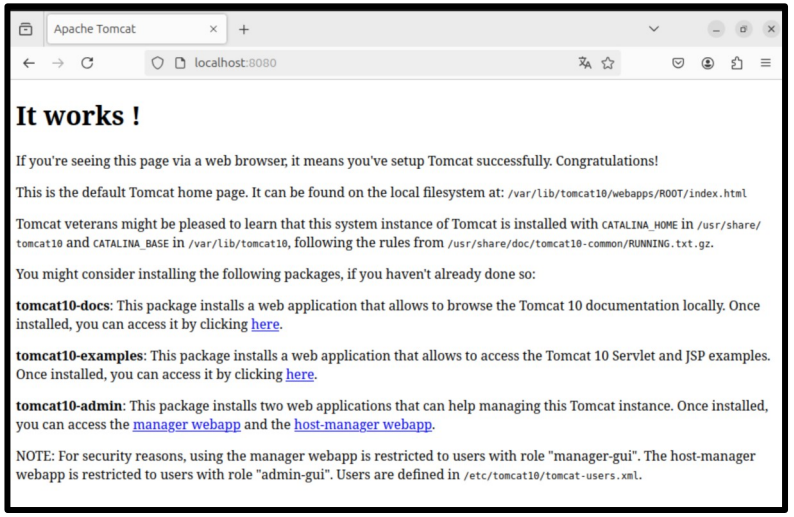


CATALINA_HOME indica el directorio de instalación de Tomcat (donde se encuentran, por ejemplo, los binarios, el programa en sí).
CATALINA_BASE indica el directorio de una instancia de Tomcat (puede configurarse como CLUSTER). Si tenemos más de una instancia, CATALINA_BASE será diferente para cada una. En algunas instalaciones ambas variables de entorno apuntan al mismo directorio.

7.2. Instalar Apache Tomcat

- Ahora reiniciamos la máquina para que las variables se carguen:
  $ reboot

- La aplicación web que **Tomcat** muestra por defecto es, en realidad, una página estática que se encuentra ubicada en:
  /var/lib/tomcat10/webapps/ROOT/index.html
- A ella se accede desde un navegador web escribiendo:
  http://localhost:8080



Esto es así porque **Tomcat** usa el puerto 8080, a diferencia de **Apache** que usa el 80.

7.2. Instalar Apache Tomcat

- Por otro lado debemos saber que **Tomcat** se configura en la ruta */etc/tomcat10*.
- En esta versión 10, **Tomcat** se **instala como servicio** y esto, entre otras implicaciones, hace que se inicie automáticamente al arrancar Ubuntu.
- Mira el **apartado 7.5.** para saber como iniciar, para, resetear y comprobar el estado del servicio.

Tema 1. Implantación de arquitecturas web.

6. Servidores de Aplicaciones.

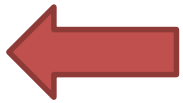
6.1. Contenedores.

7. Tomcat.

7.1. Instalando Java.

7.2. Instalar Apache Tomcat.

7.3. Instalando paquetes adicionales.



7.4. Usuarios de Tomcat.

7.5. Parar, iniciar y reiniciar Tomcat.

8. Aplicaciones web.

8.1. Estructura.

8.2. Descriptor de despliegue.

7.3. Instalando paquetes adicionales

- En este punto hemos conseguido instalar y hacer funcionar **Tomcat** pero es recomendable instalar tres paquetes; que aunque son opcionales resultan muy útiles: hablamos de la *documentación*, de los *ejemplos* y, sobre todo, de la *aplicación de administración*.

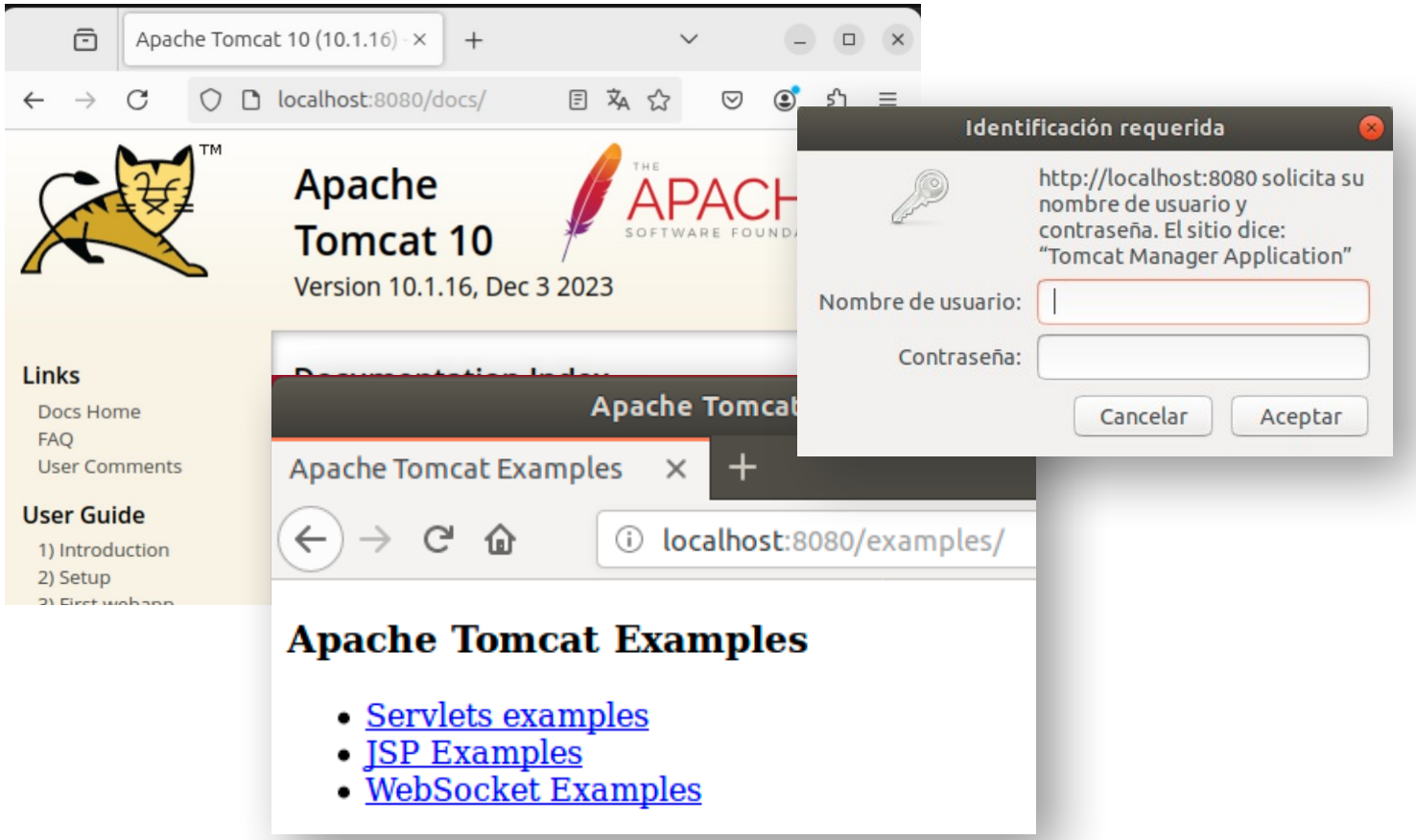
- Para ello lanzamos los siguientes comandos:

```
$ sudo apt install tomcat10-docs
$ sudo apt install tomcat10-examples
$ sudo apt install tomcat10-admin
```

Ahora podemos acceder a cada uno de ellos mediante el correspondiente enlace de la página de inicio.

7.3. Instalando paquetes adicionales

- Ahora tendremos disponible la **documentación** (en inglés), los **ejemplos** y la **aplicación de administración**:



7.3. Instalando paquetes adicionales

- Para acceder a cada aplicación escribimos en el navegador:
  - **http://localhost:8080/docs/** (Documentación).
  - **http://localhost:8080/examples/** (Ejemplos).
  - **http://localhost:8080/manager/html/** (Administración Web de Tomcat)

EJERCICIO 19.

Instala **Tomcat** y los paquetes adicionales sobre la máquina virtual creada en el ejercicio anterior. Utiliza la versión Tomcat 9 si tu sistema operativo no soporta la 10

Tema 1. Implantación de arquitecturas web.

6. Servidores de Aplicaciones.

6.1. Contenedores.

7. Tomcat.

7.1. Instalando Java.

7.2. Instalar Apache Tomcat.

7.3. Instalando paquetes adicionales.

7.4. Usuarios de Tomcat.

7.5. Parar, iniciar y reiniciar Tomcat.

8. Aplicaciones web.

8.1. Estructura.

8.2. Descriptor de despliegue.

7.4. Usuarios de Tomcat

- Para usar las aplicaciones de **Tomcat** es necesario estar registrado en el fichero /etc/tomcat10/tomcat-users.xml con un nombre de usuario y una clave.
- Al instalar los paquetes adicionales, tenemos disponible una aplicación web con la que podemos administrar el servidor Tomcat: la **aplicación manager**.
- Para poder entrar en dicha aplicación se necesita un usuario que tenga los permisos adecuados. Ello se traduce en poseer los roles *manager*, *manager-script* y *manager-gui* así como los roles *admin*, *admin-script* y *admin-gui*.
- Por tanto, editamos el archivo tomcat-users.xml e incluimos un usuario con permisos para entrar en la aplicación web **manager** *(en mi caso llamaré a ese usuario israel con clave israel)*.

```
$ sudo gedit /etc/tomcat10/tomcat-users.xml
```

7.4. Usuarios de Tomcat

- El contenido del fichero quedaría algo parecido a esto:

```
<tomcat-users xmlns="http://tomcat.apache.org/xml"
              xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
              xsi:schemaLocation="http://tomcat.apache.org/xml tomcat-users.xsd"
              version="1.0">
  <role rolename="manager"/>
  <role rolename="manager-gui"/>
  <role rolename="manager-script"/>
  <role rolename="admin"/>
  <role rolename="admin-gui"/>
  <role rolename="admin-script"/>

  <user username="israel" password="israel" roles="manager,manager-gui,manager-script,admin,admin-gui,admin-script"/>
</tomcat-users>
```

Es necesario **reiniciar** el servicio tomcat10 (apartado 7.5.)

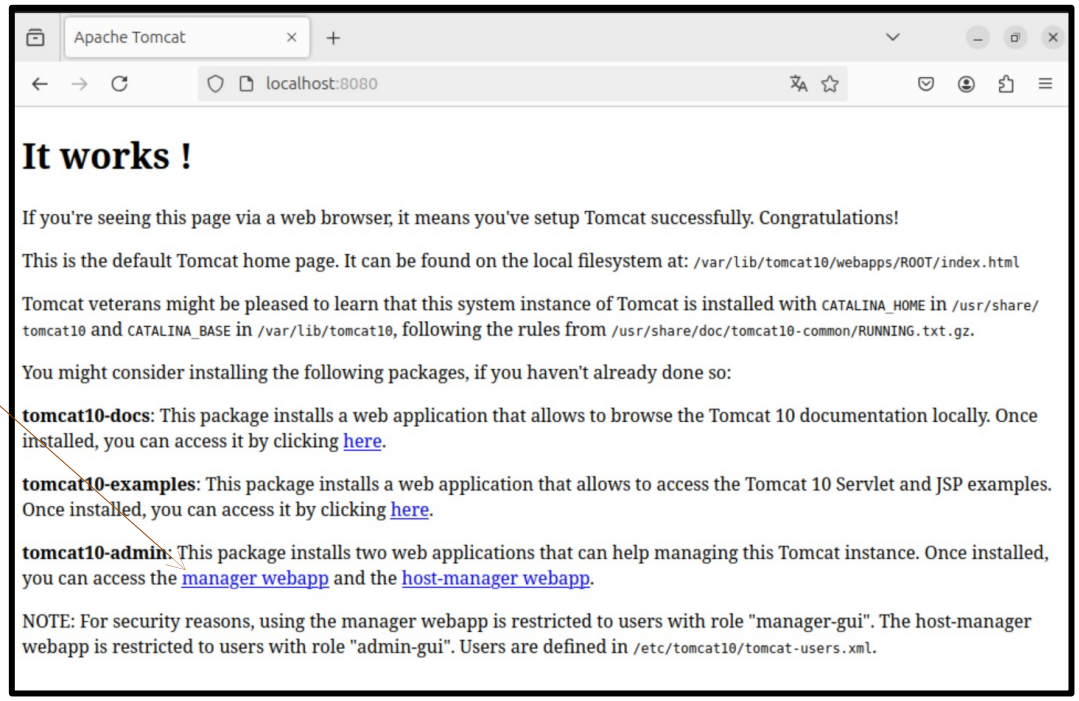
- Una vez hecho, hay que reiniciar el servidor **Tomcat**:

```
$ sudo service tomcat10 restart
```

7.4. Usuarios de Tomcat

- Ya podemos acceder a la **aplicación web manager** con el usuario recién creado **israel**.
- Lo haremos desde la página principal de **Tomcat:**

Hay un enlace directo.



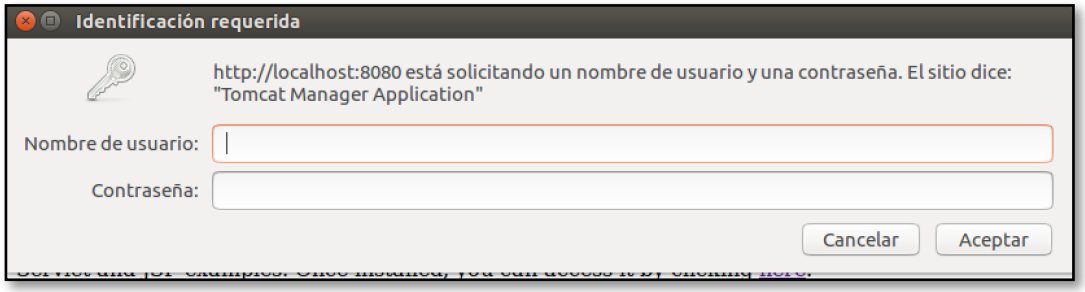
7.4. Usuarios de Tomcat

- O también podemos acceder a la **aplicación web manager** escribiendo en nuestro navegador: http://localhost:8080/manager/html/



7.4. Usuarios de Tomcat

- En ambos casos, **Tomcat** nos pedirá las credenciales para poder acceder a la aplicación **manager**.
- En nuestro caso, accederemos con el usuario israel, con clave israel; ya que solamente este usuario tiene los roles necesarios para entrar: *manager, manager-gui, manager-script, admin, admin-gui y admin-script*.

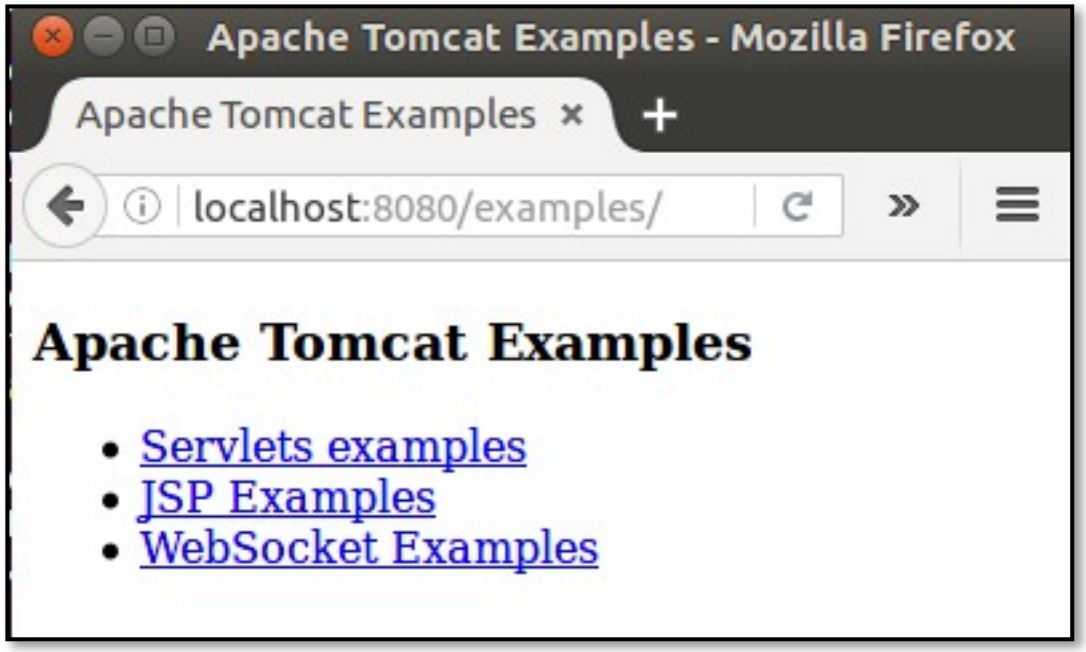


EJERCICIO 20.

Configura **Tomcat** para que puedas entrar con tu nombre de usuario y clave.

7.4. Usuarios de Tomcat

- Se recomienda también probar los ejemplos, que previamente hemos instalado con el paquete **tomcat10-examples**, accediendo a la dirección: http://localhost:8080/examples/



Tema 1. Implantación de arquitecturas web.

6. Servidores de Aplicaciones.

6.1. Contenedores.

7. Tomcat.

7.1. Instalando Java.

7.2. Instalar Apache Tomcat.

7.3. Instalando paquetes adicionales.

7.4. Usuarios de Tomcat.

7.5. Parar, iniciar y reiniciar Tomcat.

8. Aplicaciones web.

8.1. Estructura.

8.2. Descriptor de despliegue.

7.5. Parar, iniciar y reinicar Tomcat

- Podemos **parar**, **levantar** y volver a **reiniciar** el servidor **Tomcat** usando el servicio tomcat10 con los parámetros *stop*, *start* y *restart*, respectivamente:

```
$ sudo service tomcat10 stop
$ sudo service tomcat10 start
$ sudo service tomcat10 restart
```

- Incluso podemos preguntarle por el estado actual:

```
$ sudo service tomcat10 status
```

Tema 1. Implantación de arquitecturas web.

6. Servidores de Aplicaciones.

6.1. Contenedores.

7. Tomcat.

7.1. Instalando Java.

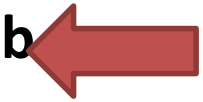
7.2. Instalar Apache Tomcat.

7.3. Instalando paquetes adicionales.

7.4. Usuarios de Tomcat.

7.5. Parar, iniciar y reiniciar Tomcat.

8. Aplicaciones web



8.1. Estructura.

8.2. Descriptor de despliegue.

8. Aplicaciones web.

- Se diferencia de una aplicación estándar en que **se accede a ella a través de una red** como Internet o una intranet.
- Se escribe en lenguajes soportados por los navegadores web (*por ejemplo Javascript + HTML*) y por lenguajes soportados por los servidores de aplicaciones (*Java, PHP, etcétera*).
- Por tanto, necesita de un navegador web para poder mostrarse y ejecutarse.
  - *Un ejemplo sencillo: un programa que permite acceder a los datos de una empresa desde fuera de ella.*
- Aquí ya queda patente el principal problema que tienen las aplicaciones web: la **seguridad**.
- Muchas veces el problema es la necesidad de encontrar un compromiso entre la seguridad y la eficiencia en la aplicación; demasiada seguridad puede *ralentizar el uso*, *aumentar el tráfico*, *etcétera*.

8. Aplicaciones web.

- **Principales desventajas:**
  - Las derivadas de depender de una red y de que accedan a la vez varios usuarios (a veces, muchos).
- **Principales ventajas:**
  - Poder usar como cliente un simple navegador web (disponible hoy en cualquier ordenador)
  - Actualizaciones más simples: no hace falta actualizar cada ordenador uno a uno.

Veamos con mayor detalle:

8. Aplicaciones web.



- **Ventajas:**
  - **No es necesario ningún tipo de** distribución, **instalación** o actualización complejo de la aplicación.
  - **Simplemente el uso de un navegador** compatible nos permitirá usar la aplicación. (*Muchas veces se crea la aplicación para un único navegador web lo que es un error. No es necesario cubrir todo el espectro pero dar al menos dos o tres opciones sería recomendable)*.
  - **No se necesitan máquinas clientes especialmente potentes**. Prácticamente cualquier ordenador en la actualidad es capaz de ejecutar un navegador web. Si la aplicación es más pesada en el cliente se pueden necesitar recursos un poco más altos.
  - Son **fáciles de integrar con otras funcionalidades de servidor** como el correo electrónico.
  - Generalmente **permiten eliminar los problemas derivados del uso de diferentes plataformas informáticas** (arquitecturas, sistemas operativos, etc.).

8. Aplicaciones web.



- **Desventajas:**
  - Generalmente las **interfaces de usuario** de las aplicaciones web son **menos intuitivas** y tienen un comportamiento peor que las clásicas.
  - Las **tecnologías web** son **muy dinámicas y cambiantes** por lo que podemos usar alguna funcionalidad que desaparezca o se modifique drásticamente obligándonos a rehacer la interfaz
  - La **ausencia de estándares en archivos** "de oficina" puede **dificultar el compartir datos e información**.
  - **Dependen totalmente del correcto funcionamiento de la red** (Internet y/o intranet).
  - Desde el punto de vista de un usuario **es preocupante la privacidad y seguridad de sus datos**. Uno de los primeros ejemplos de esto es el correo web, pero en la actualidad todas las aplicaciones de Google y muchas otras como Facebook controlan absolutamente todo lo que haces por lo que la ausencia de privacidad es notable.

8. Aplicaciones web.

- Como ya hemos comentado antes, una de las principales preocupaciones a la hora de desarrollar una aplicación web debe ser la **seguridad**.
- En muchos casos se deben proteger tanto **información crítica de la empresa** como los **datos privados de los usuarios**.
- A la hora de implementar la seguridad de una aplicación web debemos tener en cuenta cinco áreas:
  1. La **autenticación** de los usuarios: el uso de un método efectivo para asegurar que se conecten los usuarios autorizados y dificultar la suplantación de identidades es algo primordial. ¿Es una aplicación abierta a todo el mundo? ¿se pueden registrar los usuarios por si mismos?

8. Aplicaciones web.

2. La **autorización** de cada usuario: muchas veces deben existir diferentes tipos de usuarios y no todos los usuarios deben poder acceder a todos los datos o realizar todas las operaciones (*consulta, inserción, modificación o eliminación*) sobre los datos.

   La identificación y creación de diferentes roles es un aspecto a tener en cuenta desde las primeras etapas de diseño de la aplicación.

3. La **gestión de los recursos**: es necesario proteger los datos tanto cuando se encuentran en la base de datos como cuando se encuentran en tránsito entre la máquina cliente y el servidor.

   Generalmente se evitan las conexiones directas de forma remota (en una red abierta) a la base de datos. Además **es muy importante la encriptación de los datos** tanto en la base de datos como en las comunicaciones. *Los datos se utilizan como ejemplo de recurso, pero es aplicable a otros tipos*.

8. Aplicaciones web.

4. La **entrada de datos**: otro aspecto a tener en cuenta es **qué puede escribir el usuario en cada punto de entrada de datos**.

   Cuanto más limitemos los caracteres y tipo de entrada que puedan realizar los usuarios más fácil será impedir que la gente acceda a puntos no deseados de nuestra aplicación e incluso a otras aplicaciones de la misma red.

5. **Auditorias y registros**: Para poder controlar y corregir las brechas en la seguridad **es muy importante poder saber qué ha pasado**.

   Por ello debemos tener métodos de registro (por ejemplo archivos *log*) de todo lo que suceda en nuestra aplicación.

   El tamaño de los archivos debe permitirnos revisar situaciones de hace bastante tiempo ya que muchas veces un agujero en la seguridad tarda en detectarse.

8. Aplicaciones web.

- Para controlar todos estos problemas hay dos **recomendaciones** básicas:
  1. **Pruebas**: cuanto más se prueba una aplicación, menos fallos tendrá.

  Si la aplicación no es muy pequeña, es prácticamente imposible evitar todos los agujeros de seguridad pero hay que intentar minimizarlos.

  Lo ideal es tener una persona o un equipo que sepan lo que hacen intentando atacar la aplicación.

  Este es el motivo por el que muchos hackers han terminado trabajando para compañías muy potentes.

8. Aplicaciones web.

2. Usa un **marco de trabajo** (**Framework**):

Si tenemos un equipo de trabajo no queremos que cada miembro “haga la guerra por su lado”.

Todos los integrantes deben usar uno o varios métodos comunes para gestionar la seguridad de la aplicación.

Un marco común mejorará enormemente la seguridad de la aplicación.

Un marco de trabajo para aplicaciones web debe tener en cuenta los siguientes aspectos: persistencia de datos, gestión de sesiones y autenticación de usuarios, seguridad, uso de cachés, uso de plantillas (*con tipos de datos, etc*.) e incluir una interfaz de administración.

Tema 1. Implantación de arquitecturas web.

6. Servidores de Aplicaciones.

6.1. Contenedores.

7. Tomcat.

7.1. Instalando Java.

7.2. Instalar Apache Tomcat.

7.3. Instalando paquetes adicionales.

7.4. Usuarios de Tomcat.

7.5. Parar, iniciar y reiniciar Tomcat.

8. Aplicaciones web.

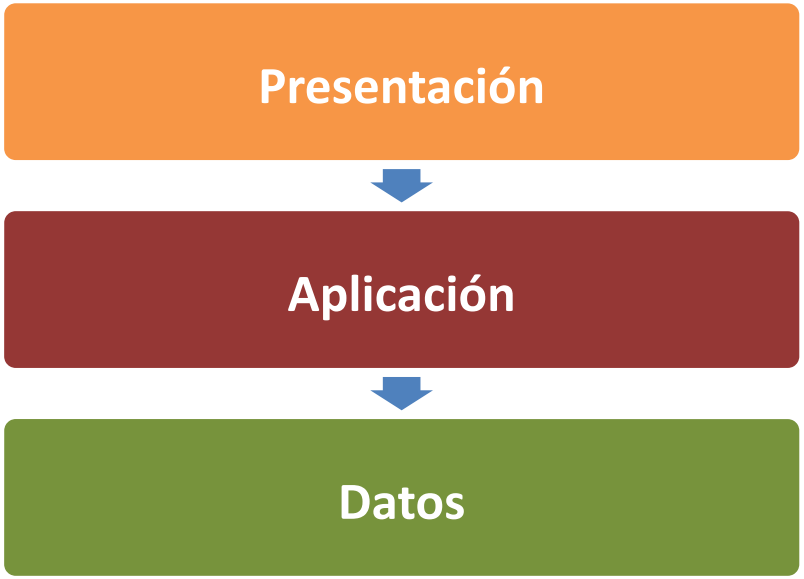
8.1. Estructura.

8.2. Descriptor de despliegue.

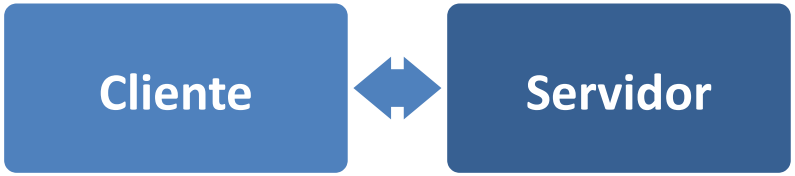
8.1. Estructura

- La estructura de una aplicación web es **equivalente a la de un servidor web** como vimos al principio del tema.
- Al tener que funcionar en todos los niveles, se organiza en capas (ver diagrama).
- La más habitual es en tres capas (*aunque la capa intermedia puede subdividirse, como ya vimos*): presentación, aplicación y datos.
- Otra forma de ver una aplicación web es en dos capas: cliente y servidor (ver diagrama).
- **Según dónde recaiga la mayor parte del trabajo, la aplicación será de cliente pesado o de servidor pesado**.

Vista en 3 capas:



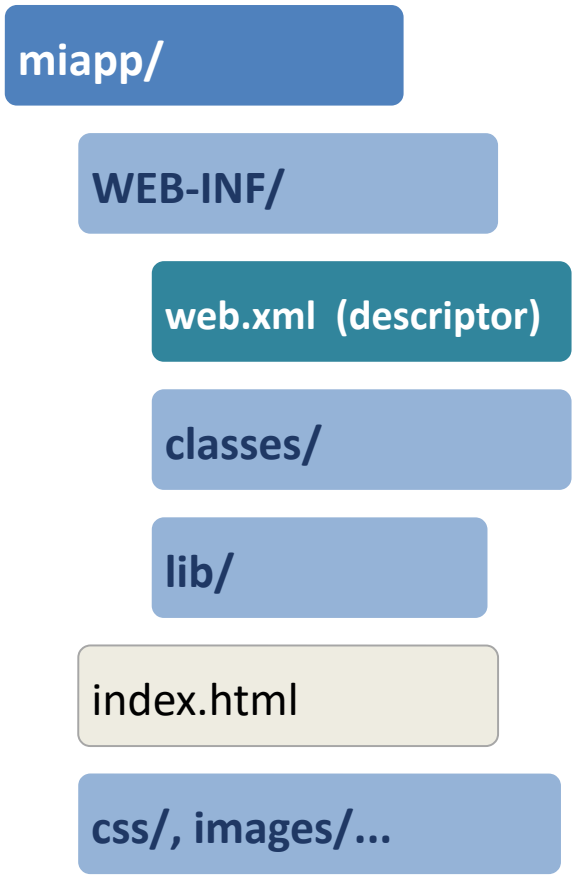
Vista en 2 capas:



8.1. Estructura

- La estructura de una aplicación web también se usa para referirse a la **distribución en directorios** de todos los elementos que componen dicha aplicación (ver ejemplo a la derecha).

Ejemplo de estructura de directorios:



Tema 1. Implantación de arquitecturas web.

6. Servidores de Aplicaciones.

6.1. Contenedores.

7. Tomcat.

7.1. Instalando Java.

7.2. Instalar Apache Tomcat.

7.3. Instalando paquetes adicionales.

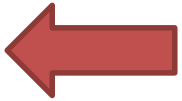
7.4. Usuarios de Tomcat.

7.5. Parar, iniciar y reiniciar Tomcat.

8. Aplicaciones web.

8.1. Estructura.

8.2. Descriptor de despliegue.



8.2. Descriptor de despliegue

- El descriptor de despliegue de una aplicación web **es un archivo de configuración en el servidor de aplicaciones**.
- En J2EE este archivo se escribe usando sintaxis XML.
- Describe cómo un componente, módulo o aplicación debe desplegarse especificando aspectos como:
  - la **seguridad**,
  - las **opciones del contenedor** y
  - **aspectos de la configuración**.

8.2. Descriptor de despliegue

- Existe además un descriptor de despliegue propio de cada servidor web.
- Por ejemplo, en **Tomcat** este descriptor se encuentra en **/etc/tomcat10/web.xml**.
- Si estamos usando un entorno de desarrollo (*por ejemplo, Eclipse*) el descriptor de despliegue lo crea el propio entorno, pero siempre es conveniente revisarlo para asegurarnos de que utiliza las opciones que queramos.

8.2. Descriptor de despliegue

- Para las aplicaciones web J2EE debe llamarse **web.xml** y ubicarse en el directorio **WEB_INF** en el directorio raíz de la aplicación.
- Se pueden consultar ejemplos más completos, en la siguiente ruta, después de instalar el paquete de ejemplos de **Tomcat** (tomcat10-examples):

  /usr/share/tomcat10-examples/examples/WEB-INF

FIN DEL TEMA 1